Khalani:
A Next-Generation Blockchain with On-Chain Proofs

The Khalani Labs Team

10 November 2025

Abstract

Khalani is an attempt to design and prototype a next-generation blockchain architecture that has the potential to eventually replace the currently dominant Ethereum-style chains. We postulate a collection of novel features for this new blockchain:

- purely functional programming on-chain
- linear types (i.e. language-level resources)
- formal logic on-chain (i.e. the ability to formulate and prove theorems and bind them with on-chain programs)
- on-chain pub-sub and timers (i.e. a proper event-based programming)
- support for intent -based programming
- code-as-asset (so that a code deployed on-chain generates financial profits and can be traded)
- account abstraction (ability to programmatically define the behavior of blockchain accounts)

We present the details of how these features fit together and how they provide remedies for known problems of existing blockchains. Although some of them are already present elsewhere, the overall vision is new. We conclude with an overview of Axi, Khalani's on-chain programming language.

Caution: this *litepaper* sketches only the key ideas from the on-going research. It is different from the upcoming Khalani *whitepaper*, which will contain detailed technical specifications.

1 Introduction

1.1 Timeline of blockchain technology

Blockchains have been around since 2009 (when Bitcoin was launched). Key milestones include:

- 2013 – understanding that decentralized money could be generalized to a decentralized computer
- 2015 – launch of Ethereum, the first widely adopted decentralized-computer platform
- 2020 – the rise of proof-of-stake consensus protocols, promising lower energy use
- 2022-2023 – wider adoption of zk-rollups (e.g., mainnet launches of zkSync Era, Polygon zkEVM), based on the idea of *zero-knowledge proofs*

As of 2025, the EVM camp – consisting of several blockchains compatible with the Ethereum Virtual Machine – is still dominant as measured by TVL (Total Value Locked).

1.2 Major obstacles for blockchain adoption

The adoption of blockchains remains limited. There are several obstacles hindering wider use.

1.2.1 Bugs are deadly yet hard to prevent

Bugs in on-chain programs are extremely costly – a single smart-contract bug, once exploited by a hacker, can easily destroy the on-chain business that depends on it. However, current methods for preventing bugs are insufficient – testing cannot guarantee the absence of bugs, and neither can audit. Moreover, once a bug is found, it cannot be fixed, because on-chain programs are immutable by design. Even if a program is explicitly designed to support an upgrade operation (for example using proxies), this upgrade usually comes too late to prevent losses caused by hackers.

Some examples:

- June 2016 – The DAO exploit ($50M lost), resulting in the subsequent Ethereum hard fork
- November 2017 – Parity wallets failure ($150M lost)
- August 2022 – Nomad bridge failure ($190M lost)

1.2.2 Scammers everywhere

Because anyone can deploy a program to a public blockchain, users face a core question: which services are trustworthy, i.e. will they actually do what they claim? How can users be confident a service will not steal their funds?

Some examples:

- December 2020 – Compounder Finance ($10M lost)
- October 2021 – AnubisDAO ($60M lost)
- November 2021 – Squid Coin ($3M lost)

1.2.3 "Lost in the maze" syndrome

Blockchain technology evolves rapidly. For typical users the space is a maze of chains, services, tokens, credentials, keys, exchanges and responsibilities which becomes increasingly hard to navigate. Although there are numerous efforts to establish community standards (for example ERC-20 tokens, wallets, DEXes), they do not offset the quickly growing complexity.

1.2.4 Cross-chain integration

According to CoinMarketCap, as of October 2025 there are 70 blockchains with market cap exceeding $1B. The ecosystem does not seem to converge to a single, universal public blockchain and the users have already got used to this diversity of independent chains. This creates integration problems, the first and foremost of which are cross-chain swaps, i.e. moving assets across blockchains.

1.2.5 The programming model is limiting

For software developers, blockchains are not nearly as convenient as more traditional computing platforms. The example limitations are:

The paid-call problem. to use a given blockchain, a users needs to acquire the native cryptocurrency of this blockchain (in order to pay gas fee). This means the gas mechanics constrains the architecture of dapps. This problem could be mitigated by features like sponsored transactions or delegation.

Explicit transaction signing. Every action taken by a user necessarily takes of form of a transaction, which in turn requires cryptographic signing. Although signing could be theoretically done by any software whenever needed, this would create a security vulnerability around private keys sharing. Therefore users tend to use a single "trusted" wallet app for all the signing work, which mitigates sharing risk but on the other hand enforces every act of signing to be a tedious manual interactive process, which in result impairs overall user experience. This could be mitigated if the blockchain programming model supported delegation.

Lack of a uniform delegation mechanism. The previous case is just part of a bigger one: delegation is just not there in blockchains. Say we have Alice (user), Bob, Charlie and Dylan (on-chain services). Bob provides some complicated functionality which involves further interaction with Charlie and Dylan. Alice has no rights to interact with Dylan, so she is forced to use Bob as a privileged intermediary. Charlie recognizes Alice as a user with certain access rights. Bob cannot directly call Charlie because Bob has no rights to do so. Hence Bob would like to interact with Charlie on behalf of Alice and this is exactly the essence of delegation. When Alice wants to ask Bob to do something for her, Bob will have troubles providing satisfactory level of "automation" of actions on behalf of Alice. This is not because such a delegation is fundamentally impossible, rather just the blockchain industry has not agreed on sufficiently safe and universal mechanism of delegation yet. For example, Ethereum proposals ERC-4337 and EIP-7702 suggest such a universal delegation mechanism that could be added to Ethereum.

The "self-driving taxi paradox". Given the currently available autonomous driving technology, combined with wireless Internet access, it is possible to run a taxi service without a driver. However such a taxi business would face a seemingly trivial problem – how a car without a driver would accomplish filling itself up with gas? So here is the paradox – a self-driving taxi still needs an operator just to accomplish the gas station interaction. In blockchains we have a similar trouble: services cannot pay for their own gas consumption. Normally, it the transaction sender who pays gas for the whole transaction execution. This immediately becomes a problem when a service has a common part (say "a backend") that from time to time needs to perform some maintenance tasks. These tasks are not performed for a specific user of the service, these tasks are for the service as a whole. But because the service has no way to cover the gas cost of these maintenance activities, the developer of the service has only two solutions available:

1. charge random users during their interaction
2. invoke maintenance whenever needed using a dedicated "housekeeper "account.

Unfortunately 1. is not reliable (this approach can blow a transaction because of exceeded gas limit) and not fair, while 2. is even more unreliable because it introduces a dependency on a centralized service. The whole issue could be mitigated by supporting the *gas reserve* mechanism.

No timers and pub-sub Smart contracts cannot natively react to time or on-chain events. While reactive features (in particular the observer pattern) are standard in off-chain programming, they are missing from on-chain programming. Because of this, developers have to route reactive processing via off-chain observers, which weakens decentralization.

Limited privacy Everything on the blockchain is anonymous but public, which is not at all the same thing as privacy. This significantly limits the development of services that critically depend on the privacy of customers' actions.

1.2.6 The open source paradox

Software distribution largely falls into two camps:

- **Commercial software** must be purchased, which funds its creation and maintenance.
- **Open-source software** is free. It is created and maintained by enthusiasts in their spare time.

The open source paradox is this: open source creators, despite providing crucial value, are not paid for their work. An additional layer of the paradox is that a lot of commercial software depends on open-source software.

In the blockchain world the situation is even more paradoxical, as there are two dimensions of software to consider:

- node software that runs the P2P network - smart contracts that run on-chain

In theory node software can be closed source, but in practice this never happens because transparency is valued so much that only open-source really qualifies for the purpose.

As for on-chain programs, in theory they are not open-source, although they can be easily decompiled and copied because they are visible in the blockchain state. In practice, trust concerns and implied transparency requirements mean that users prefer smart contracts with published source, regardless of decompilation. Put simply: users won't trust a dapp that does not publish its smart contract source code.

In summary, blockchains lean very strongly towards open-source; monetization is not based on selling or hiding code. Founders most often try to monetize their blockchain via native-asset economics, for example through an Initial Coin Offering. But with more adoption, the blockchain starts running on its own, which means that core on-chain components are developed and maintained by a broader open-source community.

Hence the paradox: the builders who add the most value are not the ones rewarded. On-chain code amplifies this, as it is effectively public by default.

1.2.7 Integration paths are not well-established (yet)

Public blockchains are available to anyone, but integrating a blockchain into a business is difficult in practice (except for simple payments), as it is often necessary to hire developers who will design, implement, and deploy a custom dapp. Easier integration, without having to build a full dapp, requires the blockchain community to converge on convenient and widely adopted abstractions. An example of such an abstraction is intent-based processing (which we discuss later).

1.2.8 Performance

Blockchain performance is low. Throughput (measured in transactions-per-second) is constrained by networking and execution. Latency (the delay between transaction sending and its confirmation/finalization) is driven by consensus design and network limits.

1.3 Our mission

We aim to prototype a next-generation blockchain. Our focus is on the programming model, i.e. the Virtual Machine (VM). We pair a novel and innovative VM design with some existing performant proof-of-stake consensus protocol.

We hope that the adoption of Khalani will challenge EVM dominance by offering significant usability gains and that, as a consequence, its ideas will spread to other chains, inspiring innovation and moving the industry forward.

The key distinguishing features of the Khalani blockchain are:

- Axi, a purely functional programming language for smart contracts
-
  - type system based on linear logic/quantitative type theory (aka "programmable assets")
  - on-chain formal logic for proving theorems about smart contracts
  - concepts (infrastructure that combines interfaces with formal specifications, in other words combines programming and logic into a convenient flow of "semantic interfaces")
- native support for intents and delegation
- on-chain events system (pub-sub and timers)
- gas reserve mechanism
- native account abstraction
- code-as-asset (code you wrote earns money for you, code is an asset and can be traded)

The following Table 1 gives a bird's eye overview of key blockchain adoption obstacles (as listed above) vs features of Khalani blockchain that are supposed to mitigate them.

Table 1: Problems and proposed solutions

Problem	**Khalani's solution**
Bugs (exploited by hackers)	Khalani smart contracts are written in Axi, a purely functional programming language. Axi supports concepts and using formal logic to prove that a component is compliant with a concept. The existence of concepts will generate social pressure on developers, as the reputation of their dapps will depend on the compliance with well established concepts. The source of the pressure will come from users because the concepts infrastructure will allow easy to use guidance for average users (similar to how easily people use SSL in internet browser).
Scams and frauds	Code in a purely functional programming language is very transparent, making it harder to obfuscate what a smart contract actually does. Formal logic provides an effective way to express formal guarantees. Checked proofs are part of the blockchain state agreed upon by consensus. This means that every user could perform a basic smart contract audit by checking whether it implements a few traits that are commonly understood as providing some anti-fraud protection.
Lost in the maze syndrome	Intents
Cross-chain integration	Intents
Delegation mechanism is missing	Delegation mechanism
Self-driving taxi paradox	Gas reserve
No on-chain timers	In-protocol support for timers
No on-chain pub-sub	In-protocol support for pub-sub
Open source paradox	Code-as-asset
Integration paths not well established	Economic incentives to build and converge around common abstractions (concepts), implemented in Axi as concepts.
Performance	UTXO consensus, which is based on partial order (as opposed to a sequence), with declared inputs of every transaction. Also, the object ownership model reduces the collisions between transactions.
Limited privacy	For now the relevant features of Khalani are not decided yet. We are looking into existing solutions (such as Monero and ZCash) to estimate feasibility.

1.4 Sources of inspiration

The Sui blockchain was the main inspiration behind our ledger structure and the use of linear types to guide the programming language's interaction with the ledger. We also extensively studied UTXO-based blockchains (chiefly Nervos and Cardano) to understand their advantages and limitations.

When designing Axi, our guiding principle was Intuitionistic Type Theory. The logical features were inspired by proof assistants such as Coq and Lean, whereas the programming side drew inspiration from languages like Haskell (the general spirit) and Unison (regarding optimal blockchain-aware source code representation). Earlier we rejected an approach based on Denotational Proof Languages and the Athena proof system.

We also took some time diving into the current state of intent-based processing ideas:

- `https://eips.ethereum.org/EIPS/eip-7521`
- `https://anoma.net/`
- `https://docs.essential.builders/welcome/`

2 Ingredients of the architecture

2.1 Axi

2.1.1 Functional programming with formal logic built-in

In **imperative programming**, the most widespread programming paradigm, the role of the program is to modify the computer's memory. The program is composed of a bunch of statements. The most basic statement is assignment, whose role is to set a variable to a particular value and a variable is a name that refers to a particular location in memory. These basic assignments are combined into more complex programs using other statements, like loops and branching. Execution of the program proceeds statement by statement, at every step modifying tiny chunks of memory. When the execution is complete, the transformation of the computer's memory described by the program has been carried out.

Functional programming is a programming paradigm in which the role of the program is to compute a single output value. The entire program is an expression which, in turn, is composed of a bunch of subexpressions. The two most basic kinds of expressions construct anonymous functions and apply them to arguments. Variables are names that refer to a particular expression and they are immutable. Simple expressions are combined into more complex ones using yet more expressions, like pairing or pattern matching. Execution of the program proceeds by transforming the program according to a set of rewrite rules that dictate how the various kinds of expressions interact with each other. When the execution is complete, all that remains of the program is a single value which is the program's result.

Functional programming is less familiar to most programmers than imperative programming (which subsumes object-oriented programming), but has one significant advantage: it is much easier to reason about functional programs. In particular, it is much easier to prove theorems about such programs, including formally proving that they are correct according to some specification.

We adopt purely functional programming in Khalani precisely for this reason – because we believe this is the ultimate way to ensure correctness of on-chain programs. Axi, our programming language, was designed from scratch to precisely fit in with Khalani's ledger model and its various novel features.

Axi borrows key ideas from type-theoretical proof assistants like Coq, Lean and Agda. These languages are purely functional (like Haskell) but with an added superpower – a developer can also represent formal mathematical theories with them. So in Axi, one can implement programs and also prove theorems about these programs.

2.1.2 Concepts

Axi merges programming and logic via **concepts** ("lawful traits"). A concept bundles together declarations of constants, functions, abstract types, and logical statements about them. In other words concepts is how the profits of formal logic are transported to the programming layer.

Concepts can capture high-level on-chain concepts such as DEX, bridge, swap, and escrow. We expect a growing library of such concepts to develop organically over time, thus encouraging convergence on common abstractions needed for integration.

2.1.3 Linear types

Linear types enforce that a value is used exactly once, thus allowing the type system to precisely track resources. This lets developers:

- safely deallocate an object after its use
- design software interfaces that guarantee a resource cannot be used after it has been closed or transitioned to a different state

While brewing since the '80s, linear types have been popularized around 2012 when they appeared as the core concept in the Rust programming language. In 2018 this idea was then adopted by the Move programming language, where linear types are the way to keep track of **resources** during program execution. This neatly fits into the on-chain money abstraction.

Move underpins Sui and Aptos, demonstrating the power of linear typing for building blockchain abstractions. This is why we decided to adopt linear typing in Axi.

2.2 Blockchain model

2.2.1 UTXO and state evolution

At the general level, Khalani blockchain is based on the UTXO model. This means that global history is composed of cells and transactions:

- cells represent pieces of state
- transactions represent units of computation

The model can be sketched with the following pseudocode:

```
struct Cell {
  address: Address,
  datatype: Datatype,
  version: Long,
  data: Bytes,
  status: CellStatus
}

struct Datatype {
  package: Package,
  module: String,
  structure: String
}

enum CellStatus {
  Alive,
  Dead
}

struct Transaction {
  nonce: Nonce,
  inputs: Vec<(Address, InputMode)>,
  outputs: Vec<Address>,
  txid: Hash
}

enum InputMode {
  Dependency,
  Update,
  Consumption
}
```

The blockchain state evolves by committing transactions. Each transaction consumes input cells and creates output cells. Cell identity is retained by its address. Each address has a linear version history. `Dependency` is a special read-only input mode in which the cell is not killed.

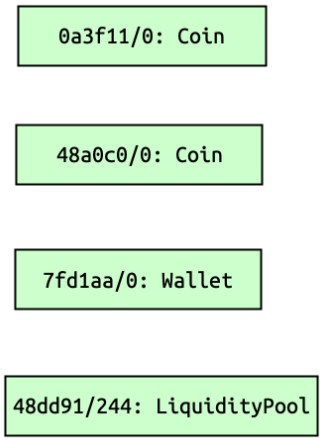
Cells read with input mode `Consumption` switch status to `Dead`. Cells read with input mode `Update` also switch status to `Dead`, and a modified copy (same address and type, incremented version) is created among the outputs.

Every cell is typed (with an Axi type) and contains exactly one value of that type. The type is set when the cell is created and cannot change during its lifetime. In other words, updates preserve the cell's type.

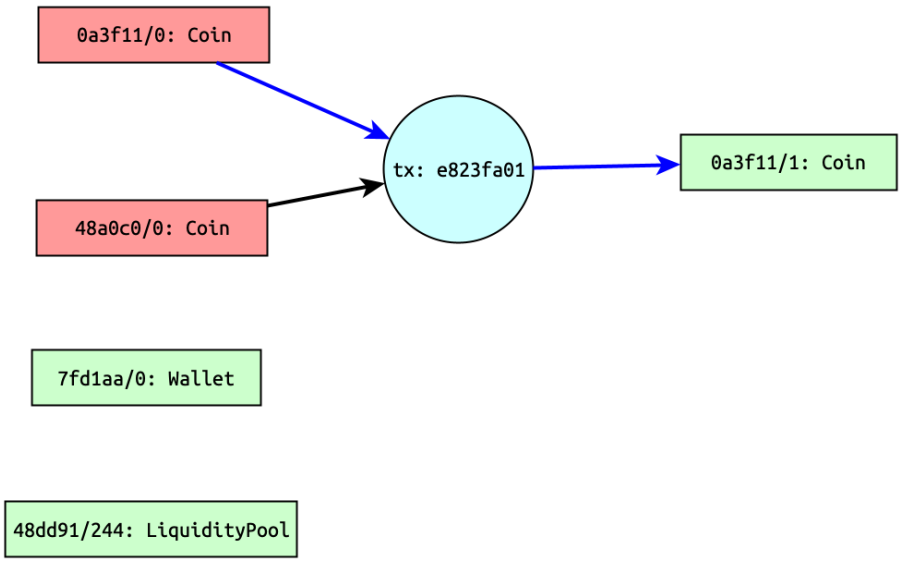
To illustrate the intuition behind the model, in the pictures we use the following conventions:

- 3-byte hashes as cell addresses - 4-byte hashes as transaction identifiers - `0a3f11/1` denotes version `1` of the cell at address `0a3f11` - cell types are shown as simple names - alive cells are green - dead cells are red - solid black lines represent cell consumption/creation - solid blue lines represent cell update - dotted lines represent dependencies (i.e. `input-mode=Dependency`)

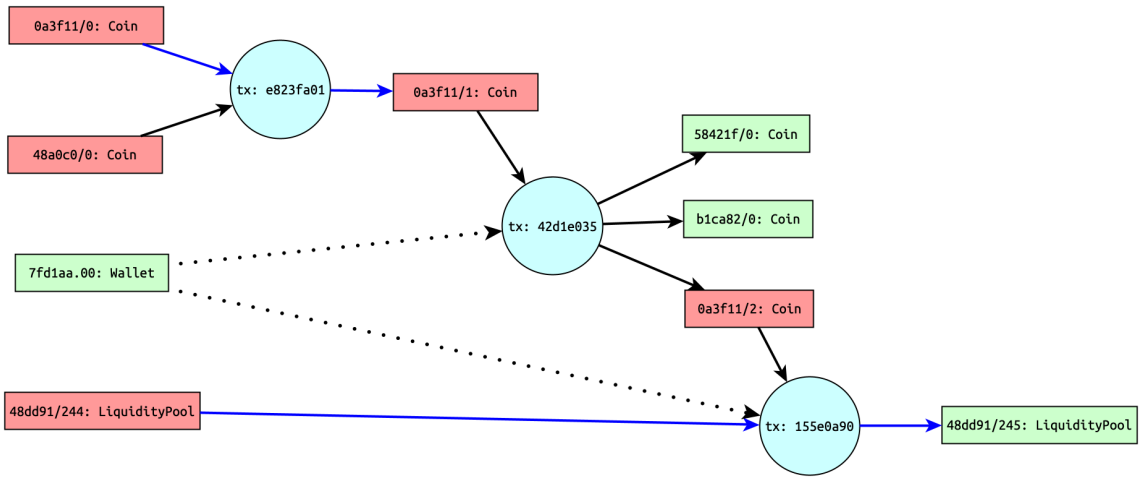
Suppose we start with four live cells:



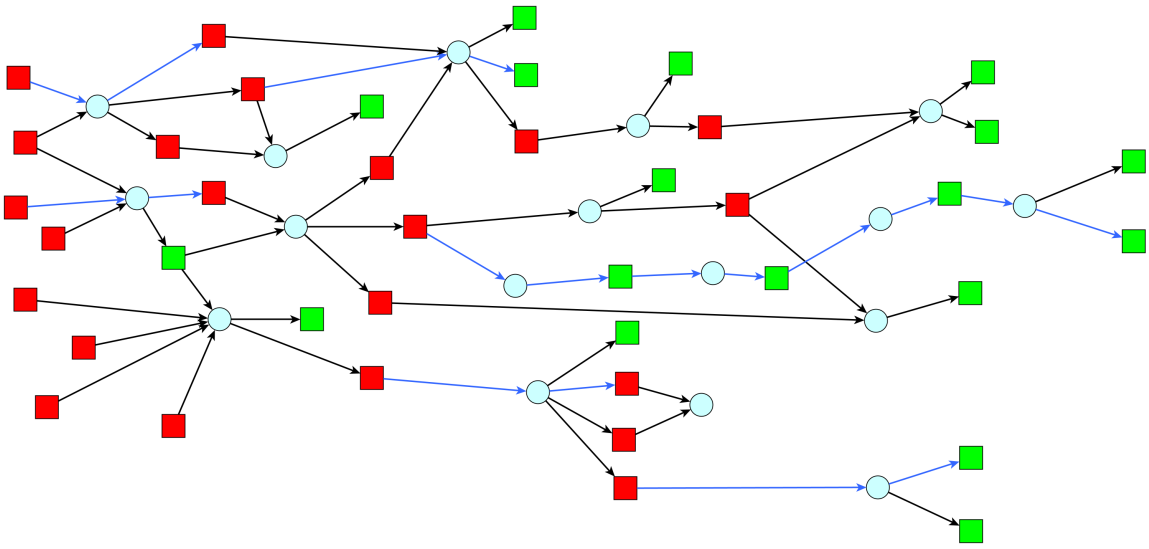
Then transaction `e823fa01` is executed. It has two inputs and one output. Cell `0a3f11` gets updated (so a new version of it is created). Cell `48a0c0` gets consumed (i.e. destroyed):



Then transaction `155e0a90` is executed. It has two inputs and one output:



The ledger grows indefinitely, forming a Directed Acyclic Graph (DAG) which might look like this:



The set of live cells is the current global state. It represents the latest state of the blockchain's memory.

2.2.2 Packages, modules and source code

On-chain code is organized into packages and modules. Every code deployment creates a new package. A package contains a hierarchy of modules (identified by their hierarchical names) that serve as namespaces and contain Axi code: types, constants, functions, traits, theorems and proofs.

The source code of a package is stored on-chain as an abstract syntax tree (AST) whose nodes are content-addressed by cryptographic hashes, so duplication is impossible. The package itself is addressed by the hash of its contents.

Every package is public and an import system allows code from any package to use code from any previously deployed packages. In other words, a package is a public shared library.

Packages are immutable and permanent, i.e. once deployed, they cannot be changed in any way and they stay on-chain forever. Thus there is no package "upgrade" mechanism and no package versioning. Deploying a modified version of an existing package yields a new independent package with a new hash, regardless of name.

2.2.3 Accounts and ownership

As in other blockchains, Khalani has the concept of user accounts. An account is established as a public-private key pair, obtained within a selected cryptosystem. Khalani supports a pluggable architecture of cryptosystems in use.

Account address is obtained by SHA-3-256 hash of the concatenation of two values: cryptosystem id (1 byte) and the public key from the key-pair generated by the user.

Every transaction must be signed by an account. The address of signing account is call "tx sender".

Changes done by a transaction to the blockchain state are limited by access control mechanics. Every cell is either shared or address-owned. In the address-owned case, the owner may be an account or another cell.

2.2.4 Where are the smart contracts?

The reader may be surprised we have not mentioned smart contracts yet, while they otherwise seem to be an ubiquitous concept in the blockchain industry. This is for a good reason.

Historically, the common understanding of what a smart contract is, has been established in the technological context of Ethereum. Ethereum has two types of accounts (we ignore here the latest ERC-4337 addition of so called "smart accounts"):

- externally owned accounts (EOA)
- smart-contract accounts (SCA)

The model of Ethereum was heavily inspired by object-oriented programming ideas, so the above separation can be explained in the following way:

- the blockchain is a computer
- EOAs represent users (and are just cryptographic key-pairs, used for signing transactions)
- SCAs represent programs (and operate as if being standalone mutable objects)

Inside a SCA data and code operating on this data are bundled together, i.e. exactly like you would have in a singleton object in any object-oriented programming language. Because public methods of a SCA form an API that other SCAs (and transactions) can call, it can also be seen as a small "server" operating on-chain.

However in most object-oriented programming languages, classes and instances are separated, so that classes contain code, while objects contain data. This way, a class serves as a template and several instances of it (objects) can be created, all of these objects sharing the same behavior. This abstraction is not properly reflected in the Ethereum model. This problem can be easily fixed, though – it would be sufficient to detach code from objects, i.e. have objects and classes as separate concepts in the VM. But when you follow this idea, the concept of a "smart contract" does not fit to the situation anymore.

This is exactly what we do in the Khalani blockchain:

- code is in modules
- data is in cells
- every cell is really an object – its "class" is the type it points to

Because every cell points to its type, cells correspond to objects in the object-oriented model. Features of a cell are defined in the module which defines the cell's type.

In summary: smart contracts from Ethereum can be ported to the Khalani blockchain – every smart contract would then correspond to a cell. We just do not use the term "smart-contract" for a cell because the code defining the structure and behavior of the cell is not part of the cell (instead it is in the corresponding module). We also do not want to use the term "smart-contract" meaning the code inside of a module, because this could create even bigger confusion for people used to Ethereum model (because in Khalani the closest thing to Ethereum smart contracts are really (shared) cells).

2.2.5 Cell usage patterns

The blockchain cell-based model is very flexible and it is not desirable (or even possible) to enumerate up-front all the software patterns that developers will come up with over time. But we wanted to mention three of such patterns:

- service
- asset
- mandarin

A **service** is a pattern that effectively resembles smart-contract accounts from Ethereum: there is an on-chain object sitting at a fixed well-known address and offering a public interface to be called by other on-chain actors (objects and accounts). And example of such service could be a DEX, a bridge, a name-registration service, an oracle or a facade to on-chain economical game.

An **asset** pattern is for creating generally "passive" resources which can be traded/purchased: cryptocurrencies, NFTs, wrapped coins, soul-bound tokens. An built-in cryptocurrency of Khalani is also materialized using asset pattern.

A **mandarin** pattern is the one we promote for intent-based processing. We introduce it later in the document.

2.2.6 Consensus

We have not decided yet which consensus protocol will be used in the Khalani blockchain. Most likely this will be a variant of proof-of-stake, which takes into account the native concurrency of the UTXO model.

The UTXO model is a hypergraph, where transactions are hyper-edges. The computational dependencies between transactions have the structure of a partial order. What the consensus protocol has to do is to achieve byzantine agreement across blockchain nodes in regards to this partial order. An example of such protocol is Mysticeti:

`https://sui.io/mysticeti`

2.3 Delegation

2.3.1 Actors on a blockchain

Before we even delve into the actual delegation problem, we need to discuss the more general problem of "actors on a blockchain". In the traditional IT actors are processes (in a single computer) or users (in a security domain) or computers (in a network). But what are actors on a blockchain?

On Ethereum, possible actors are accounts and smart-contracts. Both can own money and both can transfer money. This situation gets a little different in Khalani, for two reasons:

- It turns out the distinction between accounts and smart-contracts generates a collection of problems. A general umbrella term for the attempts to solve these problems is "account abstraction". On a blockchain with account abstraction support the distinction between accounts and smart-contracts mostly evaporates – we discuss this later in the **Account abstraction** chapter. And we do want to have account abstraction in Khalani blockchain.
- In the Khalani blockchain model, objects (i.e. live cells) play the role of smart-contracts from Ethereum. But they are kinda more flexible: in Ethereum smart-contracts are just like standalone objects with their API that can be called, while in Khalani objects can be owned, so they can be used both as services and as resources, actually being both at the same time. It is a little like in quantum physics, where light is neither a wave nor a particle, but if you insist to talk about light using the previously established abstractions of particles and waves, then light is both at the same time.

Hence we end up with the conclusion that in Khalani blockchain actors are just objects. Objects can own and can be owned, so they can serve all purposes at once.

Therefore when we apply the well known cryptography convention to use names Alice, Bob. Charlie, Dylan as the actors of a protocol in consideration, we generally think of them as on-chain

objects. It is just enough to remember that these objects are flexible enough to represent account, services and resources.

2.3.2 Introducing the delegation problem

Generally speaking, on a blockchain everything is permission-based. In a basic model of accounts (like on Ethereum) access rights are implied by 3 rules:

- only the transaction sender can transfer ether out from his account
- only the smart contract itself can read/write its instance variables
- only the smart contract itself can transfer ether out from its balance

These rules seem simple and logical but they turn out to be quite limiting when it comes to implementing delegation patterns.

Delegation is a family of patterns based on a simple common idea:

> Alice has permission to perform action X. She has however some reasons why she does not really want to perform X herself. Rather, she wants to ask Bob to perform X on her behalf. For this to be possible, the permissions of Alice are delegated to Bob.

Ideally we want the delegation mechanism to be maximally restrictive, so that the scope of Bob using Alice's permissions is maximally narrow (so that Bob cannot abuse the delegation mechanism against Alice's will).

The lack of native mechanism of delegation in Ethereum leads to various troubles, generally pushing developers to creating workarounds like this one:

`https://eips.ethereum.org/EIPS/eip-2612`

In the meantime, some attempts to introduce general-purpose universal delegation mechanism are underway (ERC-4337 and EIP-7702).

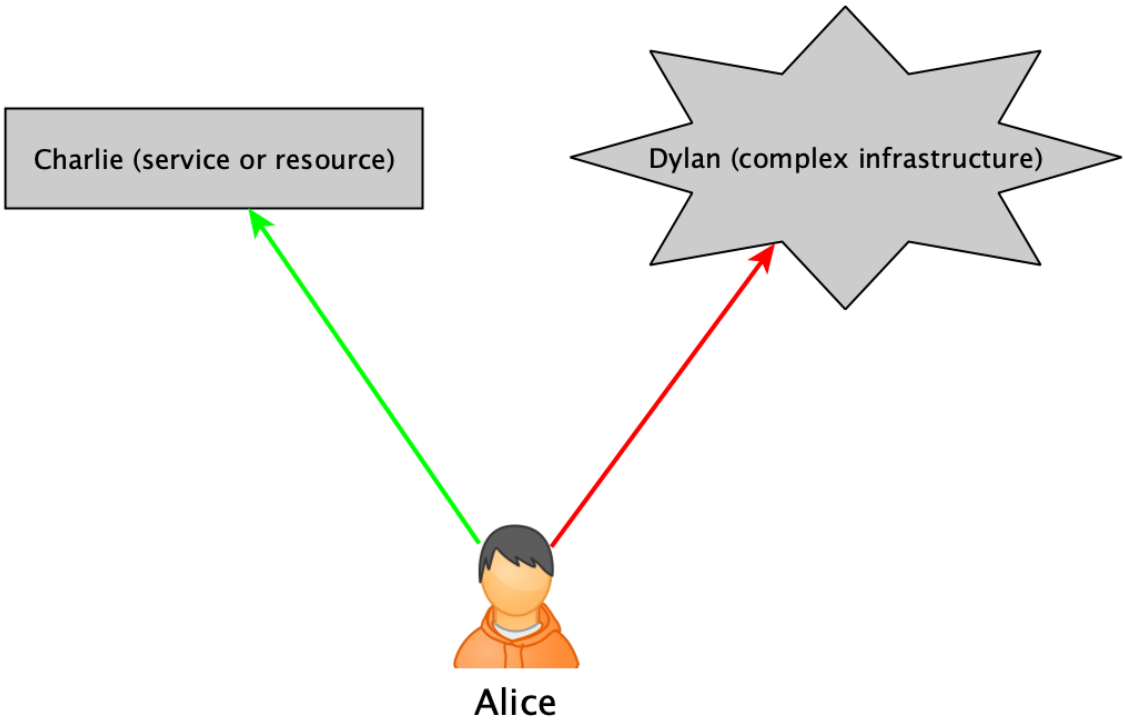
2.3.3 A motivating example

Let's look closer at the example situation we sketched in the Introduction chapter:

> Say we have Alice (user), Bob, Charlie and Dylan (on-chain services). Bob provides some complicated functionality which involves further interaction with Charlie and Dylan. Alice has no rights to interact with Dylan, so she is forced to use Bob as a privileged intermediary. Charlie recognizes Alice as a user with certain access rights. Bob cannot directly call Charlie because Bob has no rights to do so. Hence Bob would like to interact with Charlie on behalf of Alice

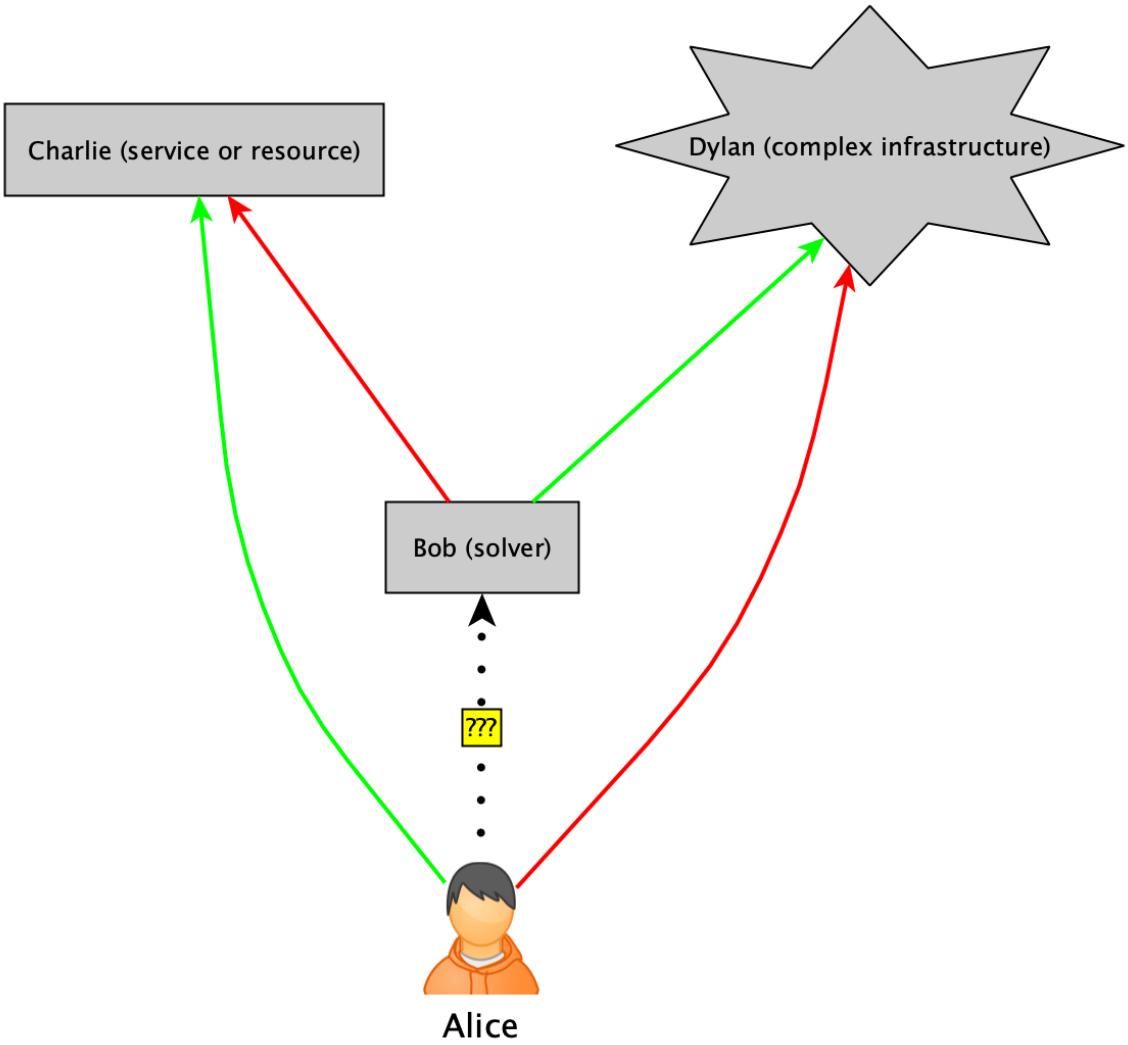
On the pictures below we use green arrows to denote "has access" and red arrows to denote "access denied".

So we start with Alice who has a problem:

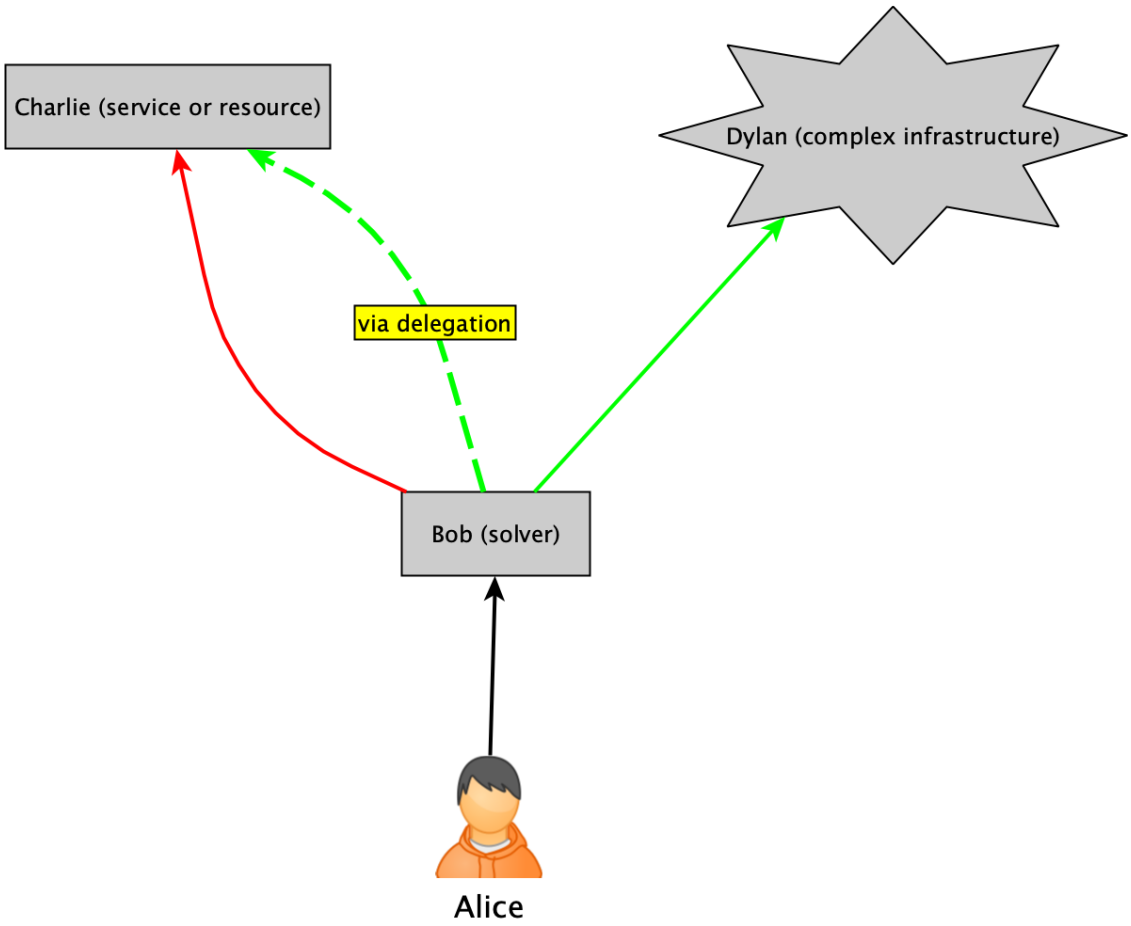


Alice want to achieve certain business goal. However, for this to happen, Alice would need to use both Charlie and Dylan to achieve her goal. Charlie is a service or resource which Alice has access to. Dylan is a complex or access-restricted infrastructure, which is beyond Alice' reach.

Alice wants to solve her problem. She found a helper service Bob (hence we named it "a solver"), who can help her with the complicated task of accessing Dylan infrastructure:



The missing element here is the ability to **delegate** Alice's access rights. She wants to temporarily lend Bob her ability to access Charlie, so that Bob can solve her problem.



2.3.4 Delegation in Khalani blockchain

The delegation mechanism is potentially very powerful. In particular it is a crucial element of the infrastructure needed for intent-based processing (which we explain later, in the **Intents** chapter). At the same time, any such mechanism is extremely dangerous, because by its very nature it is there to bypass access-rights limitations, hence malicious players can utilize this as a a universal fraud tooling.

In Khalani, we attempt to get best of both worlds: maximal safety and maximal flexibility. Therefore, we do not follow the Ethereum strategy to introduce delegation. Instead, our delegation model is based on a variant of the capability pattern. We allow Alice to form a so called "speculative handle" to represent her rights to access Charlie. Alice can pass such a handle to Bob so to execute the delegation of privileges. However, this delegation is non-exclusive (i.e. can be given to several solvers) and it also can be revoked. That being said, the mechanism is still under research and we have not decided what will be its final form.

2.4 Pub-sub, timers and gas reserve

2.4.1 Pub-sub on-chain

Publish-subscribe is a well established programming pattern:

`https://en.wikipedia.org/wiki/PublishâĂŞsubscribe_pattern`

In most mainstream blockchains every smart contract can emit events, but the observers can only be off-chain agents. Such version of pub-sub is of course useful, but it has severe limitations. For example if used for the solvers reputation system (for the context what are solvers reputation systems and why they are needed, go to the **Intents** chapter) then the off-chain relayer is needed so to pass events to an on-chain reputation registry. This relayer then happens to become a centralization

point, undermining trust. This is especially troublesome given the fact that the solvers reputation system is all about establishing the trust levels for solvers.

The escape from such circular problems is simple when a blockchain supports on-chain pub-sub mechanics:

- every smart contract can announce events
- every smart contract can subscribe/unsubscribe to events
- subscription involves handler registration
- when an event is emitted, all the registered handlers are invoked

From the developer's perspective this can be captured as follows:

```
type EventHandler<T> = fn(String, data: T);

pub trait EventEmitter<T> {
    fn subscribe(&self, symbol: String, handler: EventHandler<T>);
    fn unsubscribe(&self, symbol: String);
    fn emit(&self, symbol: String, data: T);
}
```

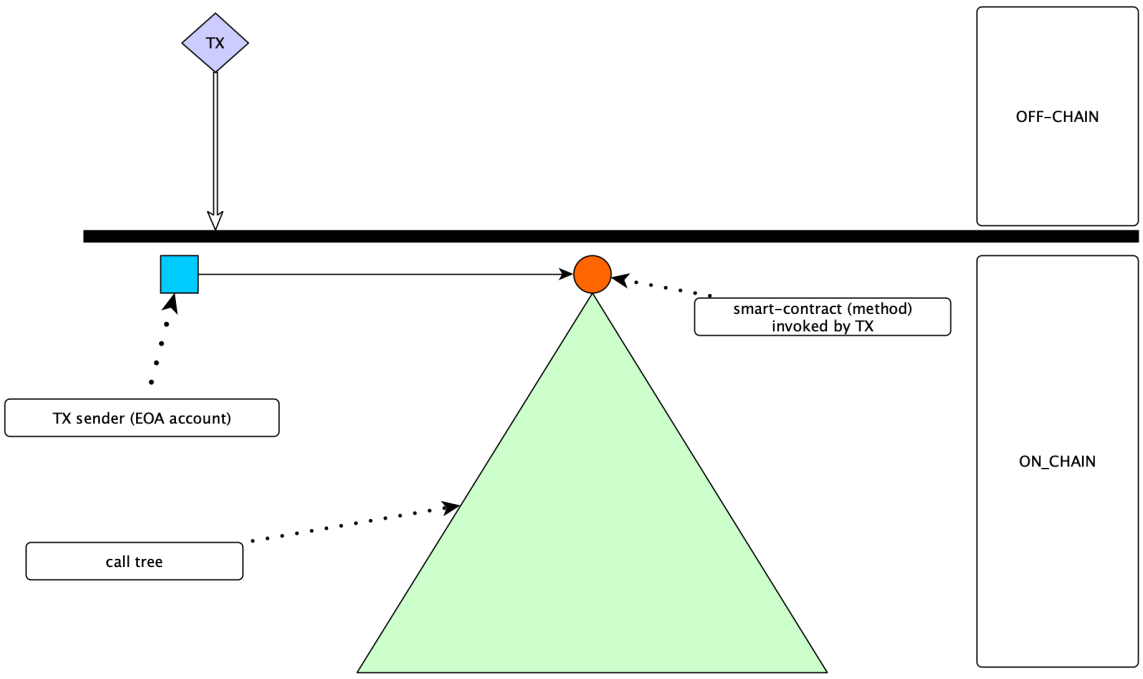
However the semantics of on-chain events is more tricky than it is for traditional Ethereum-like event systems with off-chain observers only. The new problem here is: who pays gas for the handler's execution? We discuss this problem in the next chapter.

2.4.2 Gas reserve

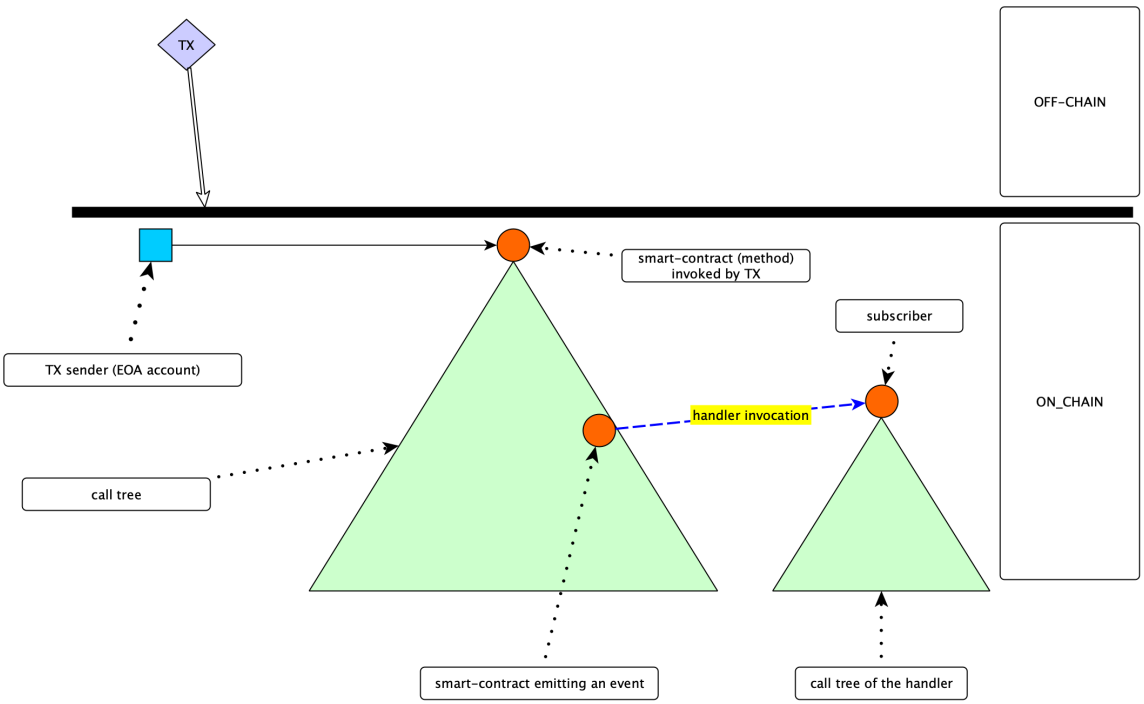
Obviously we cannot charge the event emitter – this would be against the very idea of events: emitter only announces things and does not expect any penalty for this.

Therefore it seems logical is should be the subscriber who pays the gas. Now it becomes clear why Ethereum does not support on-chain pub-sub: the desired gas charging semantics would fundamentally break the gas model of Ethereum.

Without pub-sub, the payment model is simple: the whole call tree gas cost is covered by the balance of the sender account:



With pub-sub present, the situation gets more tricky. The invocation of an event handler is not really a "call", because it is the pub-sub machinery doing this call. Hence every handler invocation spawns a separate call tree.



We previously said that is should be the subscribed who pays the gas for the handler's execution. But even this is tricky. It could be the:

- the subscriber – understood as the smart-contract that registered the handler
- the subscriber – the sender of the transaction who invoked handler registration

Both approaches turn out to be somewhat disappointing. Big part of the problem is that gas prices are fluctuating over time. What the system should do if the gas cost of the handler execution is not sufficiently covered? The only reasonable option is to rollback the whole handler invocation. Unfortunately, such an approach makes the whole pub-sub abstraction too non-deterministic to be useful.

A safer deterministic design can be obtained if we combine the following rules:

- "best effort" subscriptions and "guaranteed" subscriptions are distinguished
- "guaranteed" subscriptions are one-off (i.e. get cancelled after one handler call)
- "guaranteed" subscriptions are covered with gas reserve (see below)
- the handler registered during a guaranteed subscription must follow the static gas regime (see below)
- explicit **gas reserve** is maintained by every smart contract (or Gas is turned into separate cryptocurrency)
- static gas estimation is supported (for sufficiently limited sub-language of the native smart-contracts language)

2.4.3 On-chain timers

Timers are almost like pub-sub, but it is the clock who triggers the invocation of a handler. Of course for this to be possible, the concept of "clock" must be somehow available on-chain. In the simplest approach the time is represented as the height of the current block.

Anyway, the question of who pays for timer handler's execution leads to similar considerations as it was in the case of pub-sub.

2.4.4 Transactional approach to pub-sub and timers

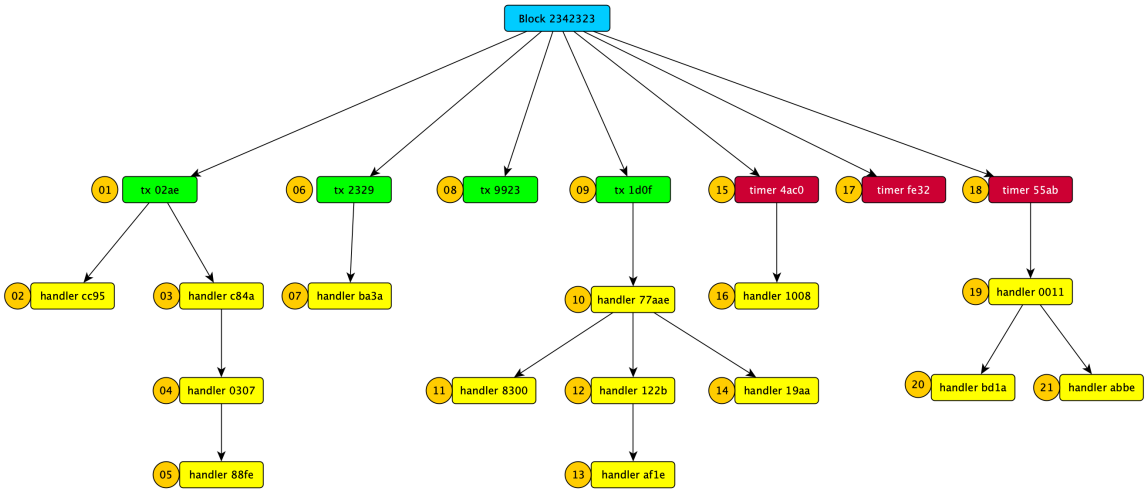
The traditional approach to covering gas cost is: the execution cost of the whole call tree is paid by the transaction sender. However – as we argued above – this model breaks down when calling handlers: someone else is to pay for the handler. It leads to an interesting question: if the call tree of a transaction A invoked a handler B (effectively another call tree) and during the execution of B's call tree the corresponding gas/cryptocurrency reserve has been exhausted – what should happen? Of course the execution of call tree B has to stop immediately. But the state of the blockchain cannot stay inconsistent – so we need to do a rollback. But because an event emitter must be shielded from any errors during the execution of handlers, it leads to the conclusion that we need to rollback call tree B only, while the A's call tree will stay intact.

The simplest way to achieve this isolation and to retain the original gas semantics is to assume that *handler invocation is executed as a separate transaction.*

In other words on a blockchain with on-chain pub-sub we would have 3 types of transactions:

- external transaction
- internal transaction (pub-sub handler invocation)
- internal transaction (timer's handler invocation)

External transactions are sent to nodes by clients. Internal transactions are added by the VM on-the-fly during the execution. The only remaining question is the ordering. The outcome of the consensus protocol is the linear ordering of blocks, and linear ordering of (external) transactions inside of blocks. When the block is executed, necessary internal transactions will be created and inserted into the original sequence of external transactions. This process can be illustrated is the following tree:



following tree:

1. Colors:
   (a) green – external transactions (ordered by the consensus protocol)
   (b) yellow – pub-sub handlers invocations

   (c) red – timers handlers invocations

2. While executing any transaction, the VM accumulates handler invocations launched by event broadcasters. Every such handler invocation will be then executed in a separate (internal) transaction.

3. Because any on-chain code can emit events, this means that any transaction (internal or external) can spawn new transactions by emitting events. This leads to a tree where nodes are transactions and edges are handler invocations. Because the whole block has the same blockchain time, timer handlers are placed at the end of a block (after all the external transactions). Their ordering should be implied by the ordering of respective subscriptions.

4. Ordering of child nodes is implied by the order of handler invocations:

   (a) for different `emit()` invocations, the order of resulting internal transactions follows from the ordering of `emit()` calls (on-chain code is executed sequentially)

   (b) for a single `emit()` invocation, the order of resulting internal transactions follows from the order of subscriptions

5. Ordering of all transactions in a block is achieved as the variant of depth-first-search traversing of the tree, where every parent mode is visited before its child nodes.

6. This algorithm ensures the following properties:

   (a) The ordering of external transactions as achieved by the consensus protocol is never changed.

   (b) If (internal or external) transaction T1 precedes transaction T2 then all event handlers spawned by T1 will be executed before T2.

   (c) Gas for every transaction (internal or external) is charged separately.

   (d) Execution of event handlers (including any errors) is not influencing the execution of the transaction that emitted events which spawned these handlers. In particular, rollback of a handler transaction will not influence the transaction where the corresponding event originated.

2.4.5 Khalani approach

The exact way timers, pub-sub and gas reserve will be accessed at Axi level is not decided yet. Nevertheless, we want all 3 mechanisms to be available.

Most likely we will not introduce Gas as a separate cryptocurrency, because this may lead to creation of a massive gas trading market, which in turn could undermine the reason for gas existence – it is there to achieve supply-demand balance around the processing power of the blockchain. We are leaning to a more limited gas reserve semantics, where an object can accumulate private, non-transferrable gas reserve for own use only and the gas price to be paid when buying gas for the reserve is $P\%$ bigger than the gas price of the current block. The fluctuation of the parameter P would then be subject to decentralized management by validators voting or it will be controlled by some in-protocol mathematical formula depending on the total value of gas burned in last N blocks vs current total amount of gas reserve.

Also, because we want provability and this involves proving behaviors that depend on events, the static-gas sublanguage of Axi will be delineated. The VM will be able to compute the gas cost of a function written in the static-gas sublanguage of Axi by static analysis. Static-gas sublanguage can be then used to implement any handler that is part of critical functionality, so that a correct gas reserve can be allocated upfront.

2.5 Intents

2.5.1 Intent-driven processing

Intent-driven processing is a novel movement in the blockchain industry which postulates a new form of interaction with blockchains, where instead of sending a transaction (or a collection of

Table 2: Concepts related to intent markets

DEX concept	Corresponding concept in i-markets
order	intent
orderbook	intentbook
order execution	filling
order type (limit order, market order, stop order...)	intent type

transactions), a user only specifies the boundary conditions for a desired outcome. A specification of such boundary conditions is otherwise known as an **intent**. The act of bringing the blockchain state to the shape defined by an intent is referred to as **filling this intent**. Software agents that fill intents are referred to as **solvers**.

An intent based ecosystem is then formed, where:

- customers publish intents - solvers observe incoming intents and engage when they can see a business opportunity - matching between customers and solvers is done either directly via the native intents support or via a suitable "intents market"

Intent-driven processing can be seen as a far-reaching generalization of the decentralized exchanges ecosystem, where intents generalize trading orders and intent markets generalize DEXes.

2.5.2 Intent-based processing vocabulary

- **Intent**: a specification of the desired effect/state of the blockchain; an intent is created by a user and usually has a form of a digitally signed document
- **Filling an intent**: materializing the desired outcome (as specified by an intent); this is the work that a solver does
- **Intentbook**: registry of intents (usually stored on-chain)
- **Solver**: a software agent capable of filling (some) intents
- **Solving**: overall activities done by a solver (includes participating in counterparty selection protocols, filling, settlement and off-chain activities)
- **Counterparty selection**: a process of deciding which solver will be given the privilege to fill a given intent
- **Settlement**: the final processing of an intent where the successful filling is confirmed and the reward is paid to the selected
- **Reward**: certain amount of money (paid in on-chain assets) that the selected solver gets for filling an intent
- **Fiasco**: the situation where the winning solver failed to fill an intent before the specified deadline
- **Rollback**: the process of returning the funds to the intent's creator (if any funds were actually locked) after fiasco

2.5.3 Intent markets

We do not want to hardcode a fixed intent-based ecosystem into the Khalani blockchain. Rather, we believe the right way to approach the problem is to ensure that a blockchain provides a sufficiently versatile toolbox of mechanisms needed for building such ecosystems.

We anticipate that the future of intents is going to be primarily driven by the concept of **intent markets**, for which we introduce the new term "i-market". An i-market is a far-reaching generalization of a Dex (see Table 2).

In particular, every Dex is an i-market.
It is the responsibility of an i-market to define:

- representation of intents
- types of intents it can handle

Some i-markets can just provide intent filling service by themselves. Other i-markets can delegate solving work to dedicated 3rd party services (solvers), hence becoming broker-like services, with the primary responsibility of connecting intent originators (customers) and intent fillers (solvers).
In the most extensible form an i-market can be:

- solver-pluggable (i.e. allowing any solver to join)
- intent-types-extensible (i.e. allowing new intent types to be defined and used)

For the intent-type extensibility we anticipate that the crucial abstraction – **mandarin** – is needed. A mandarin is a service-like cell that orchestrates the processing of a single intent. In other words, a mandarin is a self-executing trading agreement, established by parties involved in a trade. Mandarin lifecycle corresponds to the execution process of the corresponding trade.
Typically, the involved parties that use a mandarin for orchestrating a trade will be:

- intent creator (i.e. "customer")
- solver (i.e. "service provider")

In parallel with the adoption of intent-based trading we should observe an increase of well-known mandarin types deployed on-chain (with their specifications and proofs). There will be dedicated mandarins types for an extensive list of trading situations, for example:

- simple token swaps
- cross-chain token transfers
- NFT items swaps
- conditional swaps
- asset-lending agreements
- various derivatives (such as options)
- conditional investing

But this is just the beginning. Over time, off-chain service providers will find their way to use intent-capable blockchains as decentralized integrators of off-chain services. Currently we have a large collection of such integrators, just operating as centralized services, for example:

- Amazon (integrator of on-line shops)
- Booking.com (integrator of hotel services)
- Airbnb (integrator of short-term apartment renting services)
- Uber (integrator of people transportation services)

Over time, intent-based markets will appear as a viable alternatives to such service integrators. This in turn will go in parallel with completely new mandarin types – corresponding to apartment renting intents or transportation intents. Ultimately, thanks to the intent-based infrastructure, blockchains will become perfect platforms for integration of off-chain services (because there is no provider lock-in and provider bias – unavoidable in the case of centralized integrators).
An implementation of a fully-pluggable i-market has to address the following topics:

- maintaining the database of intents (i.e. the intentbook)

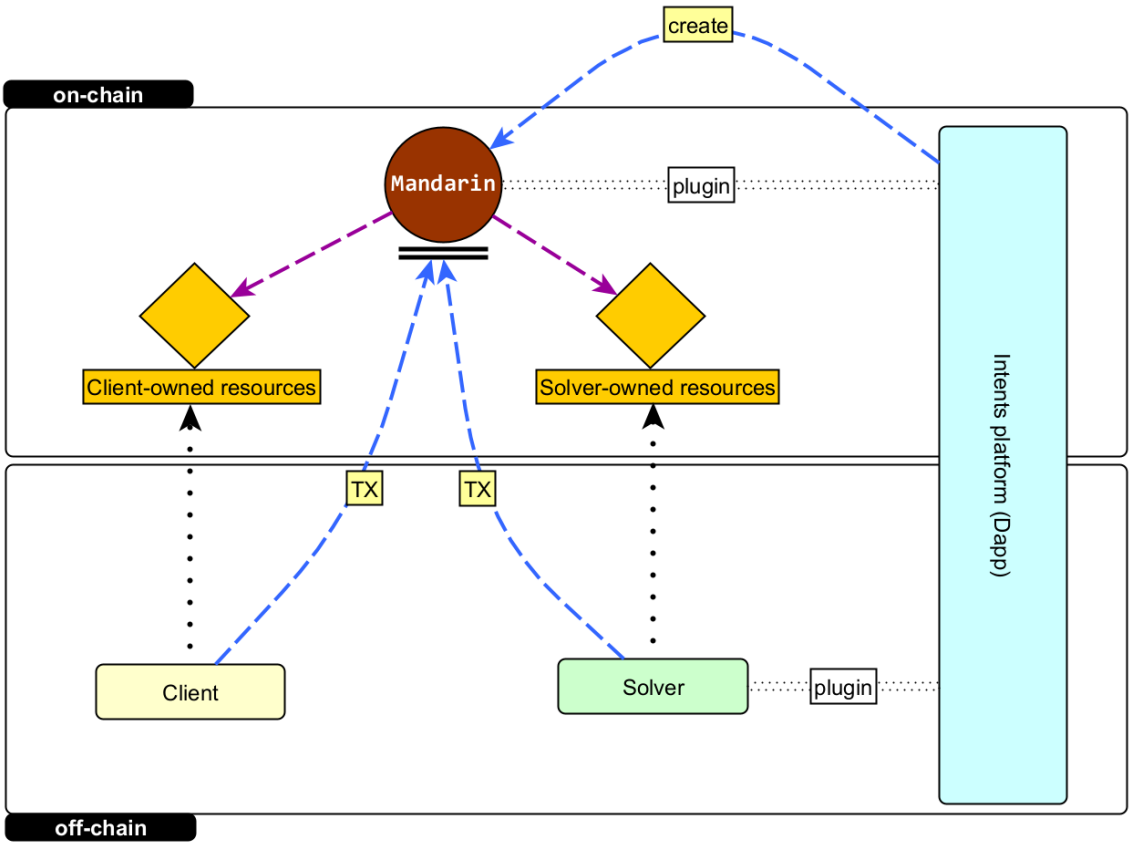
- counterparty selection (possibly several strategies)
- mandarins creation
- solvers backend (so that solvers can connect to the market and be informed about incoming intents)
- reputation infrastructure for solvers

2.5.4 Mandarins

As we stated before, a Dex is also an example of an i-market. But in a Dex all intent types (traditionally they call them "order types") are just hardcoded. There is a handful of them implemented inside of a Dex and this is it.

In the upcoming intent-based processing wave the whole point is in building i-markets that are not confined by a hardcoded list of intent types. Rather, we want such an i-market to support any intent types. So we want *pluggable intent types*.

This means an i-market has to define a suitable format of an "intent-type" plugin. In Khalani we actually suggest how this can be accomplished. We call this "the mandarin pattern".



There are two layers of this pattern:

- **mandarin type** – this is the code that implements one kind of an intent (i.e. one trading pattern)
- **mandarin object (instance)** – the cell typed with a mandarin type; it orchestrates the execution of one intent

It it not necessary but probably convenient that mandarin objects (we just call them "mandarins") are created by the i-market.

A mandarin must know participants of the trade (typically these participants will be: a customer and a solver). Its lifecycle is bound to orchestrating the execution of the trade it was created for. After the execution is completed (successfully or with a failure), a mandarin dies.

Typically, a mandarin will use delegation to perform actions on behalf of the trade participants. It will also use pub-sub to announce its lifecycle events, so that the corresponding i-market can observe mandarins it created (for the purpose of solvers reputation management).

A mandarin can be seen as a far-reaching generalization of the escrow concept. From this perspective the mandarin pattern is all about allowing programmable escrows that are capable of taking actions by themselves and that are observable via on-chain events.

2.6 Account abstraction

TODO

2.7 Code as asset

TODO

Appendix A: Overview of Axi

Axi (pronounced "Axee") is the smart contract language of Khalani. It is a purely functional programming language with a strong, static, linear type system and support for formal reasoning about programs in classical higher-order logic. Among mainstream programming languages, it is most similar to Haskell (in spirit) and OCaml (syntactically). It was also heavily inspired by proof assistants based on type theory, like Coq, Lean and Agda.

In this appendix we will have a brief, not too technical look at the philosophy and language features of Axi, together with a few simple code examples.

A.1: Philosophy

Axi's architecture is unique in that the language is split into two strictly separate layers: the programming layer and the logic layer.

The Programming Layer of Axi is a purely functional language based on higher-order polymorphic lambda calculus (aka System F_ω) with quantitative typing (linear, affine, relevant and unrestricted types) and algebraic data types. It aims to make programming pleasant and comprehending smart contract code easy, while providing essential guarantees to all on-chain code:

- **Type safety**. Well-typed programs can't go wrong, i.e. they can't crash, can't perform nonsensical operations like adding a number and a string, and can't result in any other runtime type error.
- **Resource safety**. Well-typed programs can't delete resources that are not supposed to be deletable and can't create resources out of thin air nor by duplication, unless authorized by resource type's API.
- **Memory safety**. Well-typed programs can't result in memory errors, like writing to a random memory location, nor can they have memory leaks.
- **Ownership safety**. Well-typed programs can't access resources stored on ledger unless authorized by an account owner or an interaface of a on-chain service, nor can they access resources in destroyed or nonexistent ledger entries.

The Logic Layer of Axi is a language for reasoning about Axi programs in classical higher-order logic. It allows expressing arbitrarily complex mathematical properties, thus giving programmers the ability to precisely specify the business logic and behaviour of their programs. These properties can then be either tested with random inputs before deployment, checked for concrete inputs at runtime, or proved for all possible inputs using logic. The last of these, proving, provides guarantees much more powerful than those of the type system alone, while being as credible and trustworthy as the type system. The use of the logic layer is entirely optional, i.e. programmers are never forced to engage with any part of it, although competition will encourage programmers to do so in the hopes of outcompeting other programmers' smart contracts by providing stronger guarantees.

Strict separation. In Axi, logic can reason about programs, but programs cannot manipulate logical entities, like theorems or proofs, at runtime. This one-way interaction keeps the language relatively simple, while still making it possible to express almost all functional programming idioms and all possible mathematical properties and their proofs.

Guarantees about guarantees. We not only want Axi to provide strong guarantees about its programs, but we also want to provide strong guarantees about Axi itself. We plan to use the Coq proof assistant to formally verify significant portions of Axi's metatheory, including the aforementioned properties of type and resource safety, as well as the consistency of Axi's logic. We are also considering formal verification of the reference implementation of Axi. The goal of these efforts is to ensure that the guarantees that Axi provides are trustworthy and will stay so forever, as long as the consensus protocol is followed.

A.2: Language features

In this section we will take a closer look at Axi's language features.

A.2.1: The Programming Layer

Axi is, at its core, a purely functional programming language. It thus supports all the usual features of such languages:

- **Purity and immutability**. In Axi, all values are pure and immutable – they cannot be null, cannot throw exceptions and once created, they cannot be changed. Each variable has asssigned its value once (though we allow name shadowing). This facilitates reasoning about and comprehending code, thus rendering Axi smart contracts transparent and auditable.
- **First-class, higher-order and anonymous functions**. Functions are first class citizens. They can be passed as arguments to other functions, returned from other functions as results and created on the fly. Axi encourages a programming style in which data is processed by pipelines built from basic building blocks like mapping, reducing, filtering and so on.
- **Algebraic data types and exhaustive pattern-matching**. Axi makes it easy to model the problem domain with disjoint unions and operate on these through pattern matching. This allows for writing code whose many invariants are clear from the shape of the types, decluttering the code from clunky boolean flags and cascades of conditional expressions.
- **Strong static type system**. Axi's type system makes it easy to work with high-level abstractions, like merkle trees or state machines, instead of low-level primitives, like integers or byte arrays. It is lightweight, not placing too many demands on the programmer, but still catches the most egregious errors – it can easily protect programmers from confusing with each other such objects as an integer, a string, an address, a transaction hash, a token amount and a cryptographic secret.
- **Polymorphism and traits**. Axi's type system features polymorphism (also known as "generics") as well as traits inspired by Haskell's type classes (but named after Rust's traits). These features make it easy to implement data structures and functions that work for all types that support a certain behaviour, for example a hash map that works for all types of ERC-20-compliant tokens bridged from an external blockchain.

Quantitative typing. The standard features of purely functional programming is not everything that Axi has to offer. The killer feature, as far as the programming layer is concerned, are *quantitative types*. Quantitative types control the variable usage discipline: which values are allowed to be copied and which are allowed to be left unused. This aspect of Axi's type system was inspired by linear logic and its cousin, quantitative type theory, and is an improvement upon an analogous feature of the type system of Move, the programming language of the Sui and Aptos blockchains.

In an Axi program, every variable is assigned its *quantity*, which can be either: zero, one, at most one, at least one, unrestricted. The variable's quantity determines how many times it can be used in the program, with too many or too few uses resulting in a type error. For example, the typical variable holding an NFT will have a quantity of one, forcing the program to do something with the NFT before it terminates: maybe send the NFT to the transactor, maybe trade it for something else and handle that something else properly according to its quantity, or maybe send it to somebody as a gift. What the program cannot do in such a situation is forget the NFT and do nothing with it, effectively making it "lost" in the guts of the chain. It also cannot copy the NFT and have two instead of just the original one.

To make the programmers' lives easier, every type is also assigned its "default" quantity. Every variable will be assigned the default quantity of its type unless the programmer decides otherwise. An important piece of terminology: a type whose default quantity is unrestricted is called a *data type*, whereas a type whose default quantity is either one, at least one or at most one, is called a *resource type*. There are no types whose default quantity is zero. Elements of data types are called data, whereas elements of resource types are called resources.

Traits. A trait is a way of representing an interface satisfied by a type. The definition of a trait consists of a bunch of declarations of constants and functions. The programmer can make a type an *instance* of a trait by implementing the trait's constants and functions for this type.

A.2.2: The Logic Layer

Axi's logic layer consists of typed classical higher-order logic with equality whose domains of discourse are the types from the programming layer. More precisely, this means the logic has:

- **Propositional connectives and constants**. Standard propositional connectives ($\implies$, $\wedge$, $\vee$, $\iff$, $\sim$) and constants ($\top$, $\bot$) can be used to build up complex propositions from simpler ones.
- **Quantification over terms and types**. Axi supports universal and existential quantification. We can quantify over terms of a given type ($\forall x : A, \dots$) as well as over types themselves ($\forall A : \texttt{Type}, \dots$), no matter whether we're dealing with data, resources or programs.
- **Higher-order quantification**. We can also quantify over logical entities like propositions ($\forall P : \texttt{Prop}, \dots$), predicates ($\forall P : A \to \texttt{Prop}, \dots$), relations ($\forall R : A \to A \to \texttt{Prop}, \dots$), logical connectives ($\forall C : \texttt{Prop} \to \texttt{Prop} \to \texttt{Prop}, \dots$) or even quantifiers themselves ($\forall Q : (A \to \texttt{Prop}) \to \texttt{Prop}, \dots$). This ability of logic to talk not only about the underlying programming language, but also about itself, gives it great expressive power and allows it to make use of all of mathematics when reasoning about the behaviour of programs.
- **Equality**. The built-in equality relation ($=$) makes it possible to precisely reason about programs' input-output behaviour.

Proving is carried out in a proof language based on the Curry-Howard correspondence: proofs of implications and universal quantifiers are like functions, proofs of existential quantifiers and conjunctions are like pairs, proofs of disjunctions are like disjoint union constructors, etc. Negation and biconditional are handled as in intuitionistic logic – instead of being primitives, negation ($\sim P$) is a shorthand for an implication into falsehood ($P \implies \bot$), whereas biconditional ($P \iff Q$) is a shorthand for a conjunction of two implications ($(P \implies Q) \wedge (Q \implies P)$). The principle of *proof by contradiction*, which allows to prove ($(\sim P \implies \bot) \implies P$) for any proposition P, makes the logic classical and thus intimately familiar to all mathematicians.

Equality is handled as in Martin-Löf Type Theory – proved by reflexivity and eliminated with the so-called J rule, which is a principle that allows to substitute equals for equals in the remaining part of the proof. Because proofs are hard to write using these primitives, Axi provides a convenient feature called *chaining*, which allows reasoning with long chains of equations by writing down and justifying each step separately, similarly to how equality is usually handled with pen and paper. Besides equality, this feature also supports implications, biconditionals and other transitive relations.

One novel feature that distinguishes our proof language is that the proofs can be presented in two styles: either in a *proof term style*, similar to how proofs look in Agda and Idris (as well as Coq's and Lean's manually written proof terms), or in a *tactic style*, similar to how proofs usually look in Coq or Lean. The difference between Axi and languages that also support both styles, like Coq and Lean, is that in these languages each style is its own separate proof language, whereas in Axi there is a single proof language which, thanks to a clever parser, can be formatted either like terms or like tactics. This feature will make it easier to accomodate computer scientists and mathematicians familiar with either of the aforementioned languages.

Concepts – traits in logic. Last but not least, the logic can not only talk and reason about the traits from the programming layer – it also introduces the richer and more powerful construct called *concepts*. Concepts extend traits with specifications, i.e. properties expected to be true of the trait's constants and functions. When implementing a concept, the programmer must also provide proofs of the specifications. Thus concepts let programmers precisely define what is expected of trait implementations.

Appendix B: Axi syntax

```
// In this file, we will describe the basics of programming.

// This core language consists of:
// - A mechanism for defining new programs and types.
// - Base types.
// - Functions.
// - Polymorphism.
// - Records.
// - Inductive data types and recursive functions.
// - Typeclasses.
// - Proof principles needed to reason about the above.

// We will NOT describe the following:
// - Modules.
// - Monads, effects or other means of interacting with the blockchain.
// - Blockchain specific features, like constraints or pub-sub.
// - Cryptography.

// In later files, we will describe features such as:
// - Dependent types.
// - Structural records and variants (in the core language, they are nominal).
// - Termination checker.
// - Linear typing.
// - Subtyping.
// - And possibly more...

// The syntax for a definition is ‘name : type = body‘.
answer : Int32 = 42

// Type annotations at the top-level are mandatory, so the code below
// results in an error, even though it’s easy to infer the type.
missingAnnotationError = 42

// To replace a definition with its body in a proof, we can use ‘unfold‘.
theorem unfold-example : answer === 42
proof              // |- answer === 42
  unfold answer    // |- 42 === 42
  refl
qed

// In proofterm style, it doesn’t make much sense to use ‘unfold‘,
// because its results are only visible in the goal, which is only
// visible in tactic mode.
theorem unfold-example-term-style : (answer === 42) =
  unfold answer in
    refl

// To define a type synonym, we can use the syntax ‘type name = body‘.
type MediumSizedInt = Int32

// We can use such a type just as any other.
anotherAnswer : MediumSizedInt = 42

// To see that ‘MediumSizedInt‘ is considered equal to ‘Int32‘, it
// suffices to state the following theorem. If these types were
// different, we would get a type error, but the theorem is accepted
// and we can proceed to prove it.
// Also note that to ‘unfold‘ multiple definitions, we separate their
// names with commas.
theorem type-synonym-example : anotherAnswer === answer
proof
  unfold anotherAnswer, answer
  refl
qed

// The exact set of base types is left more or less unspecified.
// I imagine there will be ‘Int‘s, ‘Uint‘s and ‘Float‘s of multiple sizes,
// as well as ‘String‘s and ‘Char‘acters, together with support for their
// literals. On the blockchain side, there might be types for ‘Address‘es,
```

```
// ‘Hash‘es, etc.

// ‘Int‘s of various sizes.
int8Example  : Int8  = -100
int16Example : Int16 = 0
int32Example : Int32 = 42
int64Example : Int64 = 2 ^ 32

// ‘Uint‘s of various sizes.
uint8Example :  Uint8  = 42
uint16Example : Uint16 = 42
uint32Example : Uint32 = 42
uint64Example : Uint64 = 42

// ‘Float‘s of various sizes.
float8Example  : Float8  = 1.0
float16Example : Float16 = 2.8e13
float32Example : Float32 = 0.4e-6
float64Example : Float64 = NaN

// Characters.
charExample : Char = '\n'

// Strings.
stringExample : String = "where lambo?"

// ‘t : A‘ is a type annotation. We can use it to give a hint to make
// the type checker think that ‘t‘ has type ‘A‘, but only if it can
// check that this is indeed the case.
// As an example, we can use an annotation to make the type checker
// think that ‘1‘ is a float, even though it is ordinarily an int.
annotationExample : Float32 =
  1 : Float32

// The syntax for anonymous functions is ‘\ x : A -> e‘.
idNat : Nat -> Nat =
  \ n : Nat -> n

// To simplify an expression to its result, use the ‘simpl‘ tactic.
theorem simpl-example : idNat 42 === 42
proof             // |- idNat 42 === 42
  unfold idNat    // |- (\ n : Nat -> n) 42 === 42
  simpl           // |- 42 === 42
  refl
qed

// ‘simpl‘ doesn't make much sense in proofterm style,
// because it only affects the goal.
theorem simpl-exampl-term-style : (idNat 42 === 42) =
  unfold idNat in
    simpl in
      refl

// When a definition needs to be unfolded to perform simplification,
// ‘simpl‘ can do it.
theorem simpl-without-unfold : idNat 42 === 42
proof                    // |- idNat 42 === 42
  simpl                  // |- 42 === 42
  refl
qed

// Of course since both ‘unfold‘ and ‘simpl‘ work only to change how the goal
// is displayed, to prove the above theorem it suffices to use ‘refl‘.
theorem refl-is-enough : idNat 42 === 42
proof
  refl
qed

// We don't need to put a type annotation on the variable if it can be inferred.
idNat' : Nat -> Nat =
  \ n -> n

```

```
// Even though we did not write the annotation on the variable,
// it will be present when the definition is unfolded.
theorem no-annotation-desugared : idNat' === (\ n : Nat -> n)
proof                          // |- idNat' === (\ n : Nat -> n)
  unfold idNat'                // |- (\ n : Nat -> n) === (\ n : Nat -> n)
  refl
qed

// We can make the definition shorter by putting all arguments on
// the left-hand side.
idNat'' (n : Nat) : Nat = n

// The short definition produces the same term as the other two definitions.
theorem short-definition-desugared : idNat'' === (\ n : Nat -> n)
proof                            // |- idNat'' === (\ n : Nat -> n)
  unfold idNat''                 // |- (\ n : Nat -> n) === (\ n : Nat -> n)
  refl
qed

// When a function has more than one argument, they are surrounded
// with parentheses.
const : Int32 -> String -> Int32 =
  \ (a : Int32) (b : String) -> a

// When there are multiple arguments of the same type, they can be
// put together in a single pair of parentheses.
constInt32 : Int32 -> Int32 -> Int32 =
  \ (x y : Int32) -> x

// To define a polymorphic function, we use the quantifier ‘forall A‘
// in the function's type. Note that in the term, we don't need to bind the
// type with ‘\‘ - type abstractions are implicit by default.
id : forall A, A -> A =
  \ x : A -> x

// Just as for monomorphic functions, we can put all the arguments on the
// left-hand side to make the definition much shorter.
id' A (x : A) : A = x

// When using a polymorphic function, we don't need to provide the type
// argument explicitly - it is inferred automatically. So to apply ‘id‘
// to an argument ‘n‘, we write just ‘id n‘.
idNat-from-id : Nat -> Nat =
  \ n : Nat -> id n

// However, we can apply the type argument explicitly if we want to.
// To do so, we prefix it with the symbol ‘@‘.
idNat-from-id' : Nat -> Nat =
  id @Nat

// Alternatively, we can prefix the function ‘id‘ itself with ‘@@‘ to turn
// all implicit arguments into explicit arguments.
idNat-from-id'' : Nat -> Nat =
  \ n : Nat -> @@id Nat n

// Last but not least, note that implicit arguments can also be inferred from
// context, even when the function is not applied to any (explicit) arguments.
idNat-from-id''' : Nat -> Nat =
  id

// If we don't want to make an argument implicit, we can make it explicit
// by prefixing it with ‘@‘ in the quantifier. However, in such a case we
// need to also explicitly bind it in the definition body.
explicit-id : forall @A, A -> A =
  \ @A (x : A) -> x

// When using ‘explicit-id‘, we need to provide the type argument explicitly.
idNat-from-explicit-id : Nat -> Nat
  explicit-id Nat

// Note that the explicit argument can be annotated.
explicit-id-annotated : forall @A, A -> A =
```

```
  \ @(A : Type) (x : A) -> x

// If we want to avoid binding ‘A‘ in the body, but we still want it explicit,
// we can move it to the left of the colon.
explicit-id-args @A (x : A) : A = x

// To state a theorem about a polymorphic function,
// we can use ‘forall‘ as a quantifier in the logic.
theorem id-spec :
  forall {A} (x : A),
    id x === x
proof
  pick-any A x
  refl
qed

// Functions enjoy definitional extensionality, i.e. a function
// ‘f‘ is equal to ‘\ x -> f x‘ and this can be proved with ‘refl‘.
theorem definitional-funext-example :
  forall {A B} (f : A -> B),
    f === \ x : A -> f x
proof
  pick-any A B f
  refl
qed

// If two functions return the same result for all arguments, they are equal.
// This principle is called (propositional) function extensionality.
// Formally, ‘funext x in e‘ proves ‘f === g‘ when ‘e‘ proves ‘f x === g x‘
// in the context extended with ‘x : A‘.
theorem funext-example :
  forall {A B} (f g : A -> B),
    (forall x : A, f x === g x) —> f === g
proof                                   // |- forall {A B} (f g : A -> B), (forall x : A, f
    x === g x) —> f === g
  pick-any A B f g                      // A B : Type, f g : A -> B |- (forall x : A, f x ===
     g x) —> f === g
  assume eq                             // A B : Type, f g : A -> B, eq : forall x : A, f x
    === g x |- f === g
  funext x                              // A B : Type, f g : A -> B, eq : forall x : A, f x
    === g x, x : A |- f x === g x
  instantiate eq                        // Theorem proved!
qed

// We can use ‘let‘ bindings.
letExample (a b : Float32) : Float32 =
  let x = a + b in
    let y = a - b in
      x * y

// In case we have more than one ‘let‘ binding, we can group them in a block.
letBlockExample (a b : Float32) : Float32 =
  let
    x = a + b
    y = a - b
  in
    x * y

// We may put type annotations on variables in ‘let‘s.
annotatedLetExample (a b : Float32) : Float32 =
  let
    x : Float32 = a + b
    y : Float32 = a - b
  in
    x * y

// When we encounter a ‘let‘ binding in a proof, we can get rid of it
// with ‘simpl‘.
theorem let-computation-example :
  forall {A} (e1 e2 : A),
    (let x = e1 in e2) === (\ x -> e2) e1
proof
```

```
  pick-any A e1 e2  // A : Type, e1 e2 : A |- (let x = e1 in e2) === (\ x -> e2) e1
  simpl             // A : Type, e1 e2 : A |- (\ x -> e2) e1 === (\ x -> e2) e1
  refl
qed

// To define a record type, we need to provide its name together with
// a list of its fields and their types.
record type Point where
  x : Float32
  y : Float32

// We can access record fields using the familiar dot syntax.
dotSyntaxExample (p : Point) : Float32 = p.x

// However, presumably we can also access fields using
// ordinary function application syntax.
alternativeFieldAccessExample (p : Point) : Float32 = x p

// In case you wonder about the type correctness of the above example,
// in a record type ‘T‘ with field ‘f : A‘, ‘f‘ is a function of type ‘T -> A‘.
fieldTypeExample : Point -> Float32 = x

// As a third possibility, we can ‘open‘ a record and refer to the fields
// directly, without dot syntax or having to apply them.
recordOpeningExample (p : Point) : Float32 =
  open p in x

// We can build records with record literal syntax.
recordLiteralSyntaxExample : Point = record where
  x = 1.0
  y = 1e10

// We can also write record literals inline.
inlineRecordLiteralExample : Point = record {x = 1.0; y = 1e10}

// Records, similarly to functions, enjoy definitional extensionality.
theorem definitional-record-extensionality-example :
  forall p : Point,
    p === record {x = p.x; y = p.y}
proof
  pick-any p
  refl
qed

// Because of the above, we can prove record equality component-wise.
theorem record-equality :
  forall p1 p2 : Point,
    p1.x === p2.x —> p1.y === p2.y —> p1 === p2
proof
  pick-any p1 p2
  assume (eqx : p1.x === p2.x) (eqy : p1.y === p2.y)
  chaining
    === p1
    === record {x = p1.x; y = p1.y}
    === record {x = p2.x; y = p1.y}  by rewrite eqx
    === record {x = p2.x; y = p2.y}  by rewrite eqy
    === p2
qed

// When we need to update a record, we can use the syntax
// ‘record r with {x1 = e1; ...; xN = eN}‘.
move-x (dist : Float32) (p : Point) : Point =
  record p with {x = p.x + dist}

theorem move-x-spec :
  forall (dist : Float32) (p : Point),
    move-x dist p === record {x = p.x + dist; y = p.y}
proof
  pick-any dist p
  apply (instantiate record-equality with _ _)
  . simpl
    refl
```

```
  . refl
qed

// We can define another record type, ‘Point3D‘, whose field names overlap
// with fields names of ‘Point‘. At the top level this causes some ambiguity,
// but we will allow to disambiguate based on types or explicitly.
record type Point3D where
  x : Float32
  y : Float32
  z : Float32

// We can explicitly disambiguate between the ‘x‘ from ‘Point‘ and the
// one from ‘Point3D‘ using dot syntax.
explicitDisambiguationExample : Point3D -> Float32 =
  Point3D.x

// We can also disambiguate implicitly, based on types.
implicitDisambiguationExample : Point3D -> Float32 = x

// Any kind of conversion between these two records must be performed
// manually.
projectToXY (p : Point3D) : Point =
  record {x = p.x; y = p.y}

embedInXYZ (p : Point) : Point3D =
  record {x = p.x; y = p.y; z = 0.0}

// Records can be polymorphic, just like functions.
// We can implement pairs as polymorphic records.
record type Prod A B where
  fst : A
  snd : B

// We can also use tuple syntax for records, but only if
// the type of the record can be inferred. In particular,
// this can always be done at the top level.
tupleSyntaxExample : Prod Int32 Float32 = (42, 3.14)

// As an alternative way of eliminating records, we can
// pattern match on them (using record literal syntax).
swapRecord A B : Prod A B -> Prod B A
| record {fst = x; snd = y} => record {fst = y; snd = x}

// Since a typical record pattern repeats all the field names twice,
// like in ‘record {x1 = x1; ...; xn = xn}‘, it makes sense to have some
// special syntax to avoid this (it is a feature known as "record puns").
swapRecord’ A B : Prod A B -> Prod B A
| record {fst; snd} => record {fst = snd; snd = fst}

// Pattern matching is also allowed using the tuple syntax.
swapTuple A B : Prod A B -> Prod B A
| (x, y) => (y, x)

// We can also pattern match on a record in a ‘let‘ binding.
swapLet A B (p : Prod A B) : Prod B A =
  let (x, y) = p in (y, x)

// ‘let‘s support all pattern matching syntaxes for records.
swapLet’ A B (p : Prod A B) : Prod B A =
  let record {fst; snd} = p in (snd, fst)

// We can define data types using the keyword ‘data type‘.
// Note that we use ‘yes‘ and ‘no‘ as names of the booleans, to make it easier
// for newcomers not to confuse them with the propositions ‘True‘ and ‘False‘.
data type Bool where
  no
  yes

// We can define functions out of data types by pattern matching.
// The syntax is OCaml-like:
// ‘‘‘
// match t with
```

```
// | pat1 => expr1
//     .
//     .
//     .
// | patN => exprN
// ```
notb (b : Bool) : Bool =
  match b with
  | no  => yes
  | yes => no

// However, since a `match` is very often the first thing we'll be
// doing when defining a function, there is a shorthand for this
// situation: we omit then final `=` and start listing the branches
// right away. Only the arguments that are not bound on the left of
// the final `:` are being matched.
notb' : Bool -> Bool
| no  => yes
| yes => no

// We can reason by cases on elements of data types using `cases`.
// The syntax is analogous as for pattern matching, i.e.
// ```
// cases t with
// | pat1 => proofterm1
//     .
//     .
//     .
// | patN => prooftermN
// ```
theorem notb-notb :
  forall b : Bool,
    notb (notb b) === b
proof                       // |- forall b : Bool, notb (notb b) === b
  pick-any b                // b : Bool |- notb (notb b) === b
  cases b with
  | no =>                   // b : Bool |- notb (notb no) === no
    simpl                   // b : Bool |- no === no
    refl
  | yes =>                  // b : Bool |- notb (notb yes) === yes
    simpl                   // b : Bool |- yes === yes
    refl
qed

// In proofterm style, the above proof looks as follows.
theorem notb-notb-term-style :
  (forall b : Bool,
    notb (notb b) === b) =
      pick-any b in
        match b with
        | no  => simpl in refl
        | yes => simpl in refl

// We can match more than one argument at a time.
andb : Bool -> Bool -> Bool
| no , _   => no
| _  , no  => no
| yes, yes => yes

// Similarly, we can reason by cases on more than one term.
theorem andb-comm :
  forall b1 b2 : Bool,
    andb b1 b2 === andb b2 b1
proof
  pick-any b1 b2
  cases b1, b2 with
  | no , no  => refl
  | no , yes => refl
  | yes, no  => refl
  | yes, yes => refl
qed
```

```
// For ‘Bool‘, we can use the more traditional ‘if‘ instead of ‘match‘.
orb (b1 b2 : Bool) : Bool =
  if b1 then yes else b2

// When proving theorems about ‘if‘s, we still use ‘cases‘.
theorem orb-no-r :
  forall b : Bool,
    orb b no === b
proof                 // |- forall b : Bool, orb b no === b
  pick-any b          // b : Bool |- orb b no === b
  unfold orb          // b : Bool |- if b then yes else no === b
  cases b with
  | no  =>           // b : Bool |- if no then yes else no === no
    simpl             // b : Bool |- no === no
    refl
  | yes =>           // b : Bool |- if yes then yes else no === yes
    simpl             // b : Bool |- yes === yes
    refl
qed

// In fact, ‘if‘ is equal to the appropriate ‘match‘.
theorem if-example :
  forall (b : Bool) {A} (x y : A),
    (if b then x else y)
      ===
    match b with
    | no  => y
    | yes => x
proof
  pick-any b A x y
  cases b with
  | no  => refl
  | yes => refl
qed

// Data types can be polymorphic.
// We use the keyword ‘of‘ to declare the constructors’ argument types,
// if there are any.
data type Option A where
  none
  some of A

// For a data type ‘T‘ with a constructor ‘c of A‘,
// ‘c‘ is a function of type ‘A -> T‘.
constructorTypeExample : forall A, A -> Option A = some

// We can use ‘if‘ not only with ‘Bool‘, but with any two-constructor data type.
// Moreover, there’s a special version of ‘if‘, ‘if t is pat then t1 else t2‘,
// which acts like pattern matching with a single branch, including binding variables
// in the ‘then‘-branch of the ‘if‘.
orElse A (x : Option A) (default : A) : A =
  if x is some a then a else default

// Proving theorems about this fancy ‘if‘ is still done using ‘cases‘.
// ‘is‘ is equivalent to the following ‘match‘.
theorem is-Option-spec :
  forall {A B} (f : A -> B) (x : Option A) (a : A) (b : B),
    (if x is some a then f a else b)
      ===
    match x with
    | none => b
    | some a => f a
proof
  pick-any A B f x a b
  cases x with
  | none => refl
  | some a => refl
qed

// In case you were wondering, the full syntax for defining data types
// is as follows:
// ‘‘‘
```

```
// data type TypeName where
//   constructor1 : ArgType1_1 -> ... -> ArgType1_M1 -> TypeName
//       .
//       .
//       .
//   constructorN : ArgTypeN_1 -> ... -> ArgTypeN1_MN -> TypeName
// ```
// However, this full syntax is quite lengthy, and that's why we've
// been using shorthands so far. Let's see the full syntax in action
// and all the possible shorthands on a simple example.

// In the full syntax, we write the full types of all constructors.
data type Sum A B where
  inl : A -> Sum A B
  inr : B -> Sum A B

// Similarly to top-level definitions, we can move the arguments to
// the left of the colon.
data type Sum A B where
  inl (a : A) : Sum A B
  inr (b : B) : Sum A B

// If the constructor arguments are on the left, we can omit the
// codomain of the constructor, since it's always the same as the
// type we're defining anyway.
data type Sum A B where
  inl (a : A)
  inr (b : B)

// Last but not least, if we only want to give the types of the constructors'
// arguments, we can use `of`.
data type Sum A B where
  inl of A
  inr of B

// All the above definitions of `Sum` are the same. The only possible difference
// could be that with the arguments named, some tools might use these names for
// tasks like generating code templates or something. Or maybe they can serve as
// style guide for copilot AIs.

// An example function on sums.
swapSum A B : Sum A B -> Sum B A
| inl a => inr a
| inr b => inl b

// The proofs go throught using `cases`, as before.
theorem swapSum-spec :
  forall {A B} (x : Sum A B),
    swapSum (swapSum x) === x
proof
  pick-any A B x
  cases x with
  | inl a => refl
  | inr b => refl
qed

// Data type can be recursive (or rather, inductive).
// Lists, the bread and butter of functional programming,
// can be defined as follows.
data type List A where
  nil
  cons (h : A) (t : List A)

// There's nothing special about defining recursive functions.
// We can perform a recursive call whenever we want.
map A B (f : A -> B) : List A -> List B
| nil      => nil
| cons h t => cons (f h) (map f t)

// Proving theorems about recursive functions requires induction.
// The full syntax is very similar to `cases`:
// ```
```

```
// induction t with
// | c_1 a_1_1 ... a_1_m => e_1
//     .
//     .
//     .
// | c_n a_n_1 ... a_n_m => e_n
// ```
// This proves ‘P t‘ provided that
// ‘e_1‘ proves ‘P (c_1 ...)‘, ..., and that
// ‘e_n‘ proves ‘P (c_n ...)‘.
// However, note one special thing: for recursive arguments (like ‘t‘ below),
// the pattern becomes ‘(t & ind IH)‘, which means that we name not only ‘t‘, but
// also the corresponding induction hypothesis.

theorem map-id :
  forall {A} (l : List A),
    map id l === l
proof                        // |- forall {A} (l : List A), map id l === l
  pick-any A l               // A : Type, l : List A |- map id l === l
  induction l with
  | nil =>                  // A : Type, l : List A |- map id nil === nil
    simpl                    // A : Type, l : List A |- nil === nil
    refl
  | cons h (t & ind IH) => // A : Type, l : List A, h : A, t : List A, IH : map id t === t |- map id (cons h t) === cons h t
    simpl                    // A : Type, l : List A, h : A, t : List A, IH : map id t === t |- cons h (map id t) === cons h t
    rewrite IH               // A : Type, l : List A, h : A, t : List A, IH : map id t === t |- cons h t === cons h t
    refl
qed

// In proofterm style, the above proof looks as follows.
theorem map-id-term-style :
  (forall {A} (l : List A),
    map id l === l) =
      pick-any A l in
        induction l with
        | nil => refl
        | cons h (t & ind IH) => rewrite IH in refl

// If you think the first proof was ugly, we can also use chaining.
// Proofs that use chaining are harder to write, as there’s much
// more writing involved, but easier to read, because the result of
// all intermediate steps in the proof are readily visible. When using
// chaining, we are really performing the same proof steps as before
// (as can be seen in the ‘by‘ clauses), but additionally their effect
// on the conclusion also gets recorded, instead of being left implicit.
theorem map-id-chaining :
  forall {A} (l : List A),
    map id l === l
proof
  pick-any A l
  induction l with
  | nil =>
    chaining
      === map id nil
      === nil
  | cons h (t & ind IH) =>
    chaining
      === map id (cons h t)
      === cons h (map id t)
      === cons h t            by rewrite IH
qed

// Let’s see some more proofs by induction for the sake of examples.
theorem map-comp :
  forall {A B C} (f : A -> B) (g : B -> C) (l : List A),
    map (\ a -> g (f a)) l === map g (map f l)
proof
  pick-any A B C f g l
  induction l with
```

```
  | nil =>
    chaining
      === map (\ a -> g (f a)) nil
      === nil
  | cons h (t & ind IH) =>
    chaining
      === map (\ a -> g (f a)) (cons h t)
      === cons (g (f h)) (map (\ a -> g (f a)) t)
      === cons (g (f h)) (map g (map f t))         by rewrite IH
      === map g (map f (cons h t))
qed

// Often, we will want another name for a variant of an already existing theorem.
// A natural name for the right-to-left direction of the above theorem ‘map-comp‘
// is ‘map-map‘.
theorem map-map :
  forall {A B C} (f : A -> B) (g : B -> C) (l : List A),
    map g (map f l) === map (\ a -> g (f a)) l
proof
  pick-any A B C f g l
  rewrite (instantiate map-comp with f g l)
  refl
qed

// When we want to do recursion on an argument which is followed by more
// arguments, we can’t use our convenience syntax and must resort to the
// more primitive ‘match‘.
app A (l1 l2 : List A) : List A =
  match l1 with
  | nil      => l2
  | cons h t => cons h (app t l2)

// List concatenation is associative.
theorem app-assoc :
  forall {A} (l1 l2 l3 : List A),
    app (app l1 l2) l3 === app l1 (app l2 l3)
proof
  pick-any A l1
  induction l1 with
  | nil =>
    chaining
      === app (app nil l2) l3
      === app nil (app l2 l3)
  | cons h (t & ind IH) =>
    chaining
      === app (app (cons h t) l2) l3
      === cons h (app (app t l2) l3)
      === cons h (app t (app l2 l3))  by rewrite IH
      === app (cons h t) (app l2 l3)
qed

// Reversing a list.
rev A : List A -> List A
| nil      => nil
| cons h t => app (rev t) (cons h nil)

// Reversing a concatenation of two list results in a concatenation of
// the reversed lists in reverse order.
theorem rev-app :
  forall {A} (l1 l2 : List A),
    rev (app l1 l2) === app (rev l2) (rev l1)
proof
  pick-any A l1 l2
  induction l1 with
  | nil =>
    chaining
      === rev (app nil l2)
      === rev l2
      === app (rev l2) nil  by rewrite <-app-nil-r
  | cons h (t & ind IH) =>
    chaining
      === rev (app (cons h t) l2)
```

```
      === rev (cons h (app t l2))
      === app (rev (app t l2)) (cons h nil)
      === app (app (rev l2) (rev t)) (cons h nil)        by rewrite IH
      === app (app (rev l2) (app (rev t) (cons h nil)))  by rewrite app-assoc
      === app (app (rev l2) (rev (cons h t)))
qed

// The core language also has a simple type class mechanism.
// Type classes are more or less equivalent to records with
// type parameters, with some special syntax to make use of
// them in type signatures.
// ‘Eq‘ is a class whose types have a function that checks
// if two elements of the type are equal.
class Eq A where
  eqb : A -> A -> Bool

// We can define an instance of the class for a particular type
// by defining all the functions the class requires.
// Note that when defining the class’s fields, the syntax is the
// same as at the top-level. For example, we must give a full type
// annotation to each function, and we can use the shorthand syntax
// for pattern matching.
instance Eq Bool where
  eqb : Bool -> Bool -> Bool
  | no , no  => yes
  | yes, yes => yes
  | _  , _   => no

// We can define conditional instances that depend on the existence
// of instances of the same or other classes.
// Note that in our example, our use of ‘eqb‘ in ‘eqb x y‘ refers to
// the ‘eqb‘ for type ‘A‘ which comes from the instance of ‘Eq‘ for ‘A‘
// that our instance depends on.
instance Eq (Option A) <= Eq A where
  eqb : Option A -> Option A -> Bool
  | none  , none   => yes
  | some x, some y => eqb x y
  | _     , _      => no

// Conditional instances can have more than one dependency.
instance Eq (Sum A B) <= Eq A, Eq B where
  eqb : Sum A B -> Sum A B -> Bool
  | inl a1, inl a2 => eqb a1 a2
  | inr b1, inr b2 => eqb b1 b2
  | _     , _      => no

// Implementations of class functions can be recursive.
// Of course they can!
instance Eq (List A) <= Eq A where
  eqb : List A -> List A -> Bool
  | nil, nil => yes
  | cons h1 t1, cons h2 ht => andb (eqb h1 h2) (eqb t1 t2)
  | _, _ => no

// We can define functions which work with all types that belong to a given
// class. The syntax for a type class argument is ‘[Eq A]‘, i.e. class
// instances are put in square brackets.
// As an example, we can define a function that checks if two elements of a
// type are not equal, provided that the type supports the ‘Eq‘ class.
neqb A [Eq A] (x y : A) : Bool =
  notb (eqb x y)

// When not on the left-hand side of the colon, class instances are put in
// a ‘forall‘, also with square brackets.
neqb : forall A [Eq A], A -> A -> Bool =
  \ x y : A -> notb (eqb x y)

// Now, a question arises: what can we prove about functions that make
// use of type classes? Not much, it turns out. We might think ‘eqb‘ is
// a function for deciding equality, but in reality it could just as well
// be any random function that does nothing useful.
// In the example below, we define an instance of ‘Eq‘ for ‘Uint8‘ (one of
```

```
// our base types), in which ‘eq‘ always returns ‘no‘. There’s nothing that
// would force us to return ‘yes‘ when the arguments actually are equal.
instance Eq Uint8 where
  eqb (x y : Uint8) : Bool =
    no

// However, not all hope is lost. Besides ordinary "programming" type classes,
// there are also type classes in the logical layer, and they can force their
// member functions to satisfy various properties, which we’ll make good use of.
// By the way, just as we can define conditional class instances, we can define
// classes that can require their arguments to belong to some other class.
class LawfulEq A <: Eq A where
  eqb-spec1 : forall x y : A, eqb x y === yes —> x === y
  eqb-spec2 : forall x y : A, x === y —> eqb x y === yes

theorem eqb-diag :
  forall {A} [LawfulEq A] (l : List A),
    eqb l l === yes
proof
  pick-any A l
  induction l with
  | nil =>
    refl
  | cons h (t & ind IH) =>
    chaining
      === eqb (cons h t) (cons h t)
      === andb (eqb h h) (eqb t t)
      === andb yes (eqb t t)       by rewrite (apply (instantiate eqb-spec2 with h h) refl)
      === andb yes yes             by rewrite IH
      === yes
qed

discriminator : Bool -> Prop
| yes => True
| no => False

theorem no-not-yes :
  no === yes —> False
proof
  assume h : no === yes
  proving False
  proving discriminator no
  rewrite h
  proving discriminator yes
  proving True
  trivial
qed

theorem andb-yes-inv :
  forall b1 b2 : Bool,
    andb b1 b2 === yes -> b1 === yes /\ b2 === yes
proof
  pick-any b1 b2
  cases b1, b2 with
  | yes, yes =>
    assume _
    both refl refl
  | no, _   =>
    assume h : no === yes
    absurd
    no-not-yes
    assumption
  | _, no =>
    assume h : no === yes
    absurd
    no-not-yes
    assumption
qed

instance LawfulEq (List A) <= LawfulEq A where
  theorem eqb-spec1 :
    forall l1 l2 : List A,
```

```
      eqb l1 l2 === yes —> l1 === l2
  proof
    pick-any l1 l2
    assume h : eqb l1 l2 === yes
    induction l1 with
    | nil =>
      cases l2 with
      | nil =>
        refl
      | cons h2 t2 =>
        absurd
        no-not-yes
        assumption
    | cons h1 (t1 & ind IHt1) =>
      cases l2 with
      | nil =>
        absurd
        no-not-yes
        assumption
      | cons h2 t2 =>
        // h : andb (eqb h1 h2) (eqb t1 t2) === yes
        proving cons h1 t1 === cons h2 t2
        lemma eq-heads : h1 === h2 by
          apply eqb-spec1
          proving eqb h1 h2 === yes
          and-left (andb-yes-inv (eqb h1 h2) (eqb t1 t2) h)
        lemma eq-tails : t1 === t2 by
          apply IHt1 t2
          proving eqb t1 t2 === yes
          and-right (andb-yes-inv (eqb h1 h2) (eqb t1 t2) h)
        chaining
          === cons h1 t1
          === cons h2 t1  by rewrite eq-heads
          === cons h2 t2  by rewrite eq-tails
  qed

  theorem eqb-spec2 :
    forall l1 l2 : List A,
      l1 === l2 —> eqb l1 l2 === yes
  proof
    pick-any A _ l1 l2
    assume h : l1 === l2
    chaining
      === eqb l1 l2
      === eqb l2 l2  by rewrite h
      === yes        by rewrite (instantiate eqb-diag with A l2)
  qed

// With lawful classes, we can indeed do some proving. As an example, let's prove
// an obvious specification of ‘neqb‘.
theorem neqb-spec :
  forall {A} [LawfulEq A] (x y : A),
    neqb x y === yes<->~ (x === y)
proof                  // |- forall {A} [LawfulEq A] (x y : A), neqb x y === yes<->~ (x === y)
  pick-any A _ x y     // A : Type, LawfulEq A, x y : A |- neqb x y === yes<->~ (x === y)
  unfold neqb          // A : Type, LawfulEq A, x y : A |- notb (eqb x y) === yes<->~ (x === y)
  both
  . assume (hnot : notb (eqb x y) === yes) (heq : x === y)
                       // A : Type, LawfulEq A, x y : A, hnot : notb (eqb x y) === yes, heq : x === y |- False
qed
```

Appendix C: Code examples

In this section we will see some code examples that demonstrate how Axi works in practice. Note that Axi is not implemented yet, so that the code examples are only an approximation of what we intend to achieve, instead of being a demonstration of an existing language.

Example 1. *An example program – list functions*

```
1  data List (A : Type) : Type where
2    nil
3    cons (h : A) (t : List A)
4
5  app @(A : Type) (l1 l2 : List A) : List A =
6    match l1 with
7    | nil      => l2
8    | cons h t => cons h (app t l2)
9
10 map @A @B (f : A -> B) : List A -> List B
11 | nil      => nil
12 | cons h t => cons (f h) (map f t)
```

The above example demonstrates the basics of Axi programming. On lines 1-3 we define an algebraic data type called `List`. The type is parameterized by a type `A` and has two constructors: `nil` is the empty list and `cons h t` is the list with head `h` and tail `t`. Both `: Type` annotations on line 1 are necessary, because we need to indicate that both `A` and `List A` are unrestricted types, as opposed to linear, affine or relevant (we will see these in later examples).

On lines 5-8 we define a function called `app` which concatenates two lists. It takes three arguments: a type `A` and two lists `l1` and `l2` of type `List A`, and returns a result of type `List A`. The type argument means that the function is polymorphic. It is also marked with the symbol `@`, which means it's an *implicit* argument, i.e. when calling the function it can be omitted and will get automatically inferred. The body of the function is simple: we pattern match on the argument `l1`. If it's `nil`, we return `l2`. If it's of the form `cons h t`, we return a list whose head is `h` and whose tail is computed by the recursive call `app t l2`.

On lines 10-12 we define another function, called `map`, which applies a function `f` to every element of a list. This time, however, we use an abbreviated syntax. The two type arguments `@A` and `@B` are marked as implicit, but we don't need to write the `: Type` annotations, which get inferred automatically. The argument `f` and its type are written out in full. Then, instead of taking another argument, we say the function returns a function of type `List A -> List B`. Because we omit the equality sign, what follows in the next two lines is a definition by pattern matching on the output function's `List A` argument.

Example 2. *Quantitative typing*

```
// KhalaniCoin : Type1 is a resource type that we'll define later.

coinForgery (x : KhalaniCoin) : KhalaniCoin = coinMerge x x
//           ^                                           ^ ^
//           |                                           | |
//           \-----------------------------------\       | |
//                                               |       | |
// Type error: duplicate use of the resource 'x' -/       | |
// of type 'KhalaniCoin : Type1'                           | |
// First use -------------------------------------------/ |
// Second use ------------------------------------------/
// Hint: remove the duplicate use
// or declare the type to be in 'Type+' or 'Type' to allow it

coinBlackhole (x : KhalaniCoin) : Unit = ()
//             ^                         ^
//             |                         |
//             \--------------------\    |
//                                  |    |
// Type error: unused resource 'x' -/    |
// of type 'KhalaniCoin : Type1'         |
// Hint: use the resource in body -------/
// or declare the type to be in 'Type?' or 'Type' to allow it
```

The above example demonstrates how the type system enforces resource discipline. We will define the type `KhalaniCoin` and the function `coinMerge` in the next example. For now it only matters that `KhalaniCoin` is a linear type, i.e. `KhalaniCoin : Type1`.

First, on line 3, we try to define a function called `coinForgery`. It takes an argument of type `KhalaniCoin` called `x` and returns a `KhalaniCoin` by calling the function `coinMerge` with both its arguments being `x`. Now, `KhalaniCoin` is a linear type, i.e. by default variables of this type must be used exactly once. Because `x` is of type `KhalaniCoin` and nothing indicates otherwise, this means that the function `coinForgery` must use `x` exactly once. However, it uses it twice when calling `coinMerge`, which is a type error.

This error is caught by the type checker, which complains that `x` was used too many times. The error message also hints at some possible solutions: use `x` less (the only solution for ordinary programmers) or redefine the type `KhalaniCoin` to allow duplicate uses (this solution is only available to the type's implementer). The fact that the type system was able to catch this error is crucial, because if the function `coinForgery` passed the type checker, it would allow programmers to create unbounded amounts of `KhalaniCoin`, causing hyperinflation, which most likely was not the intention of this type's creator.

The second function, called `coinBlackhole`, takes an argument `x` of type `KhalaniCoin` and returns `()`, which is the only value of type `Unit`, which is the type with only one element. That `Unit` is the return type indicates that the function's result "doesn't matter". Because the type of `x` is linear, `coinBlackhole` must use `x` exactly once. However, it doesn't use it at all, which is a type error.

This error is caught by the type checker, which complains that `x` was not used. Similarly to the previous error message, two hints are offered: use `x` or make the type `KhalaniCoin` affine so that it allows zero or one usages (with the latter solution being available only to the type's creator). The fact that this error was caught by the type system is very important, because `KhalaniCoin`'s creator did not intend for values of this type to be deletable.

Example 3. *`Coin` trait*

```
1  trait Coin (A : Type1) where
2    coinAmount : A -> Nat * A
3    coinZero   : A
4    coinMerge  : A -> A -> A
5    coinSplit  : Nat -> A -> Option A * A
6
7  record KhalaniCoin : Type1 where
8    amount : Nat
9
10 instance Coin KhalaniCoin where
11   coinZero : KhalaniCoin =
12     record where amount = 0
13
14   coinMerge (c1 c2 : KhalaniCoin) : KhalaniCoin =
15     record where amount = c1.amount + c2.amount
16
17   coinAmount (c : KhalaniCoin) : Nat * KhalaniCoin =
18     let n : Nat = c.amount in
19       (n, record where amount = n)
20
21   coinSplit (askedAmount : Nat) (c : KhalaniCoin) :
22     Option KhalaniCoin * KhalaniCoin =
23       let n : Nat = c.amount in
24         if askedAmount <= n then
25           ( some (record where amount = askedAmount)
26           , record where amount = n - askedAmount
27           )
28         else
29           (none, record where amount = n)
```

The above example shows the definition of the `Coin` trait, which could be one way of capturing the commonalities of user-defined coins. Then we implement the type `KhalaniCoin` from the previous example and demonstrate how to make it an instance of the `Coin` trait.

On lines 1-5 we define the trait `Coin`. It takes a parameter `A : Type1`, which is a linear type that represents the type of coins. The trait has one constant and three functions. `coinZero` represents

a coin with denomination zero (but we will prefer the word "amount" instead of denomination). `coinMerge` merges two coins. `coinSplit` splits one coin into two, with its first argument being the amount of its first output and the second output being the change; there's also the possibility of failure if the amount asked is too big, indicated by `Option` wrapping the first output's type. `coinAmount` is perhaps the most mysterious – it returns the amount of the coin, but also a new coin. This is a pattern that we will see many times to come. If it returned just the amount, calling `coinAmount` would destroy the coin, because it must be used exactly once (because its type is linear), and checking the amount counts as a single use! Therefore, `coinAmount` must not only return the amount, but also a "new" coin, which we expect to be the same as the "old" coin (but this property is not guaranteed; later we will see how to guarantee it using logic).

On lines 7-8 we implement the type `KhalaniCoin`. It is a record type with a single field `amount` which is a natural number (i.e. a non-negative number which can be arbitrarily large). `KhalaniCoin` is a linear type, because we marked it with the annotation `: Type1`. Note that this is the case even though its only field, `amount`, is of type `Nat`, which is not a linear type. In other words, the programmer is free to turn data into resources if he so wishes (although he may not freely turn resources into data).

On lines 10-29 we implement the trait `Coin` for the type `KhalaniCoin`. Note that we don't need to implement the trait's constants and functions in the same order they appear in the trait definition. On lines 11-12 we implement `coinZero` – it's a record (of type `KhalaniCoin`) whose `amount` field is `0`.

On lines 14-15 we implement `coinMerge`. It takes two arguments of type `KhalaniCoin` and returns a `KhalaniCoin` which is a record whose `amount` field is the sum of `amount`s of its arguments.

On lines 17-19 we implement `coinAmount`. It takes an argument `c` of type `KhalaniCoin` and returns a tuple whose first component is a natural number and whose second component is a `KhalaniCoin`. First, on line 18, we consume the coin `c` by peeking at its `amount` field, which then gets named `n`. On line 19, we build the return tuple – the first component is just `n`, whereas the second component is a record whose `amount` field is `n`. Note that the latter is exactly the same thing as the initial coin `c`, but we cannot directly return `c` as the second component, because `c` must be used exactly once and we have already used it when checking its `amount`.

Last but not least, on lines 21-29 we implement `coinSplit`. It takes two arguments, `askedAmount` of type `Nat` and `c` of type `KhalaniCoin`. First, on line 23, we consume `c` to check its amount and store it in `n`. Then, on line 24, we check if `n` is large enough for the split to succeed. If it does, on lines 25-27 we return a tuple whose first component is a coin whose amount is the `askedAmount` (wrapped in `some` to indicate success) and whose second component is a coin whose amount is `n` minus the `askedAmount` that went to the other coin. If `n` is not large enough, we return a tuple whose first component is `none` (to indicate failure) and whose second component is a coin whose amount is `n`, i.e. a coin that is the same as the original coin `c`.

Example 4. *A simple proof*

```
1  theorem app-assoc :
2    forall (A : Type) @(l1 l2 l3 : List A),
3      app (app l1 l2) l3 === app l1 (app l2 l3)
4  proof
5    pick-any A l1
6    induction l1 with
7    | nil =>
8      pick-any l2 l3
9      refl
10   | cons h (t & ind IH) =>
11     pick-any l2 l3
12     chaining
13       === app (app (cons h t) l2) l3
14       === cons h (app (app t l2) l3)
15       === cons h (app t (app l2 l3))  by rewrite IH
16       === app (cons h t) (app l2 l3)
17 qed
```

The above example shows a simple theorem and its proof. On line 1 we see the theorem's name, `app-assoc`, which is a mnemonic for "(list) append is associative". Its statement, on lines 2-3, reads as follows: for all types `A` and for all `l1`, `l2`, `l3` which are of type `List A`, the list `app (app l1 l2) l3` is equal to the list `app l1 (app l2 l3)`. Note that here the symbol `@` marks the three

arguments as *explicit*, whereas its absence for `A` marks this type argument as implicit – opposite to the convention used in the programming layer. These details, however, won't be very important for us now.

The proof is on lines 4-17. At any point in the proof, Axi keeps track of all the *goals* that need to be solved to finish the current proof, and also of the *context* of these goals, which is the list of all local objects and assumptions we can use in the proof. When we start the proof with the keyword `proof`, we start with a single goal that is the same as the theorem statement. The code on line 5 can be read "let `A` be any type and let `l1` be any value of type `List A`". At this point the goal becomes `forall (l2 l3 : List A), app (app l1 l2) l3 === app l1 (app l2 l3)`. On line 6 we indicate that the rest of the proof proceeds by induction on the list `l1`. The induction has two cases, `nil` and `cons`, corresponding to the two `List` constructors. This means that we now have two (sub)goals, one for each case.

On line 7 we indicate the start of the (sub)proof for the first goal, i.e. the `nil` case. At this point our current goal is `forall (l2 l3 : List A), app (app nil l2) l3 === app nil (app l2 l3)`, because `l1` was replaced with `nil`, and after line 8 it is `app (app nil l2) l3 === app nil (app l2 l3)`. Note that this information is not shown in the code – displaying the goal and context to the programmer will be handled by language tools like IDE plugins. We finish the first subgoal on line 9. `refl`, which is short for "reflexivity", indicates that this equation holds trivially, just by computing the result on both sides.

On line 10 we indicate the start of the (sub)proof for the second goal, i.e. the `cons` case. On the same line we bind the arguments of `cons`, which we name `h` and `t`. We also get hold of our induction hypothesis for `t`, which we name `IH`. After line 11, the current goal is `app (app (cons h t) l2) l3 === app (cons h t) (app l2 l3)`, as `l1` was replaced with `cons h t`.

On lines 12-16, we use *chaining* to finish off the goal. It works like this: on line 13, after the equality symbol `===`, we write the left hand side of the current goal (which must be an equation), which in our case is `app (app (cons h t) l2) l3` (line 13). Then on subsequent lines we write out what this expression is equal to (again, after `===`) and provide justifications (i.e. proofs) for why this is the case (after the keyword `by`). In our case, on line 14 we indicate that our goal's left-hand side is equal to `cons h (app (app t l2) l3)`, but we omit the justification for this. We could have written `by refl`, but this is the most trivial justification possible, so Axi guesses this is what we mean when we omit a justification. Then, on line 15, we indicate that it is equal to `cons h (app t (app l2 l3))`, justifying it with `by rewrite IH`, i.e. we are making use of our induction hypothesis. This justification is crucial and cannot be omitted, because Axi cannot infer it on its own. Finally, on line 16, we indicate that our expression is equal to the right-hand side of our goal, again omitting the justification, because it is trivial. All in all, the entire chaining proves that `app (app (cons h t) l2) l3` is equal to `app (cons h t) (app l2 l3)`, which is exactly what our goal was. Thus the proof is finished, which we indicate on line 17 with the keyword `qed`.

Example 5. *`LawfulCoin` concept*

```
concept LawfulCoin (A : Type1) extends Coin A where
  noncomputable coinAmount' (x : A) : Nat =
    let (n, _) = coinAmount c in n

  law coinAmount-spec :
    forall @(x : A),
      let (_, y) = coinAmount x in x === y

  law coinZero-spec :
    coinAmount' coinZero === 0

  law coinMerge-spec :
    forall @(x y : A),
      coinAmount' (coinMerge x y)
        ===
      coinAmount' x + coinAmount' y

  law coinSplit-none :
    forall @(askedAmount : Nat) @(coin coin' : A),
      coinSplit askedAmount coin === (none, coin')
       <->
      (askedAmount <= coinAmount' coin) === false
        /\
```

```
24        coin === coin'
25
26    law coinSplit-some :
27      forall @(askedAmount : Nat) @(coin exact change : A),
28        coinSplit askedAmount coin === (some exact, change)
29         <->
30        (askedAmount <= coinAmount' coin) === true
31          /\
32        coinAmount' exact === askedAmount
33          /\
34        coinAmount' change === coinAmount' coin - askedAmount
```

The above example shows the definition of a *concept* called `LawfulCoin` and its implementation for `KhalaniCoin`. `LawfulCoin` takes as a parameter one linear type `A`, which moreover must implement the `Coin` trait. This is what it means for the `LawfulCoin` concept to *extend* the `Coin` trait. Note this distinction: the trait and its implementation are a part of the programming layer, whereas the concept and its implementation are part of the logic layer.

On lines 2-3, we define a function `coinAmount'` that is part of the `LawfulCoin` concept. This is a feature we have not mentioned before: both traits and concepts, besides declarations, can also contain (auxiliary) definitions. In our case, we need the function `coinAmount'` to avoid using the much more cumbersome `coinAmount` when stating the properties that lawful coins should satisfy. `coinAmount'` takes an argument of type `A` (of which we think as a coin, because it has the `Coin` trait) and returns only the first output of `coinAmount`, while ignoring its second output.

It might seem that `coinAmount'` ignoring the coin returned by `coinAmount` is a type error (zero uses instead of exactly one), but function definitions that belong to the logic layer (which is indicated by the keyword `noncomputable`) play by different rules from ordinary functions. Most importantly, they don't need to adhere to the resource usage discipline. In other words, auxiliary functions defined solely to be used to state theorems can freely delete or copy resources, even it were a type error for an ordinary function to do so. This is not a problem from the point of view of ordinary programs – these "logic-only programs" cannot be run (which is where the `noncomputable` keyword comes from) – they can only be reasoned about when proving theorems, so they can never do anything bad to on-chain assets.

Next, on lines 5-34, comes the main part of the concept's definition. On lines 5-7 we declare a *law* claled `coinAmount-spec` which specifies the behaviour of `coinAmount` – the second component of its output must be equal to its input. This is the guarantee missing from the `Coin` trait that we were expecting to hold. Every implementation of `LawfulCoin` will have to provide a proof that turns this law into a theorem.

Having given a specification of `coinAmount`, all the remaining laws will use `coinAmount'` only. On lines 9-10 we declare the law `coinZero-spec` which says that the amount of `coinZero` is zero. On lines 12-16, the law `coinMerge-spec` says that the amount of a merged coin is equal to the sum of amounts of the individual coins.

On lines 18-24 the law `coinSplit-none` says that `coinSplit` returns `(none, coin')` if and only if the asked amount was not smaller than `coin`'s amount and the coins are equal. However, this only specifies one of the two cases that may happen when calling `coinSplit`, so we need another law. On lines 26-34 the law `coinSplit-some` says that `coinSplit` returns `(some exact, change)` if and only if the asked amount was smaller than `coin`'s amount, the amount of `exact` is equal to `askedAmount` and the amount of `change` is equal to `coin`'s amount minus the asked amount. Together, these two laws fully specify `coinSplit`'s behaviour: if the coin's amount is enough, the coin gets split into a coin with the amount we asked and a coin which contains the "change". If we asked for too much, we just get the original coin back.

Example 6. *Implementation of `LawfulCoin` for `KhalaniCoin`*

```
1 instance LawfulCoin KhalaniCoin where
2   theorem coinAmount-spec :
3     forall @(x : KhalaniCoin),
4       let (_, y) = coinAmount x in x === y
5   proof
6     pick-any x
7     refl
8   qed
9
```

```
theorem coinZero-spec : coinAmount' coinZero === 0
proof
  refl
qed

theorem coinMerge-spec :
  forall @(x y : KhalaniCoin),
    coinAmount' (coinMerge x y)
      ===
    coinAmount' x + coinAmount' y
proof
  pick-any x y
  refl
qed

// The proofs of coinSplit-none and coinSplit-some were elided.
```

The above example shows how to implement `LawfulCoin` for the type `KhalaniCoin`. We don't need to reimplement `coinAmount'` or anything like that – this function is defined by the concept and we can use it freely. What we need to do is to replace every `law` from the concept's definition with a `theorem`. It so happens that we implemented `coinAmount`, `coinZero` and `coinMerge` correctly and also our implementations are so simple that the proofs go through just by computation (i.e. `refl`). The only theorems requiring a non-trivial amount of proving would be `coinSplit-none` and `coinSplit-some`, but we omit these proofs because they are more about natural numbers than the concept of `LawfulCoin`.

Appendix D: Interacting with the ledger in Axi

In this appendix, we will see how Axi programs can interact with Khalani's ledger. First we will discuss a greatly simplified ledger model which we'll use in the remaining code examples. It is a specialization of the ledger model described in the main text to the most basic default case. We will also look at the primitive (built-in) types and functions that represent it in Axi. Then we will give an overview of Axi's interaction with the ledger, focusing on the "lifecycle" of resources. Finally we will discuss a slightly longer example of an on-chain service, together with example transactions that set this service up and use it.

D.1: A simplified ledger model

We will use the word *ledger* to refer to the state of the blockchain. Things on the ledger have their addresses. The three kinds of addresses are *account addresses*, *package addresses* and *cell addresses* (or just addresses for short).

Account adresses are addresses that act as the identities of external actors. They allow them to interact with the blockchain, similarly to EOAs (Externally Owned Accounts) on Ethereum.

All Axi code deployed on the ledger is stored in a big codebase, divided into *packages*. Package addresses are means of locating these packages in this codebase. Packages are further subdivided into smaller units, called *modules*, which hold definitions of types, traits, constants and functions.

Data and resources on the ledger are stored in *cells*. Each cell has an address and a type, and stores exactly one value of that type. Cells also have some metadata which won't concern us for now.

Cells are either *owned* or *shared*. Owned cells have exactly one owner who is an account. Shared cells do not have an owner. Shared cells are further subdivided into *mutable shared cells*, which contain resources and can be interacted with by anybody (any account or cell) according to the interface set by the package defining the cell's type, and *immutable shared cells*, which contain data, but not resources, and can be read by everybody, but cannot be modified by anybody.

The state of the ledger is modified by transactions. Every transaction has a *transactor* and a *transaction body*. There are two types of transactions: *code deploys* and *ordinary transactions*. The body of a code deploy transaction is a package of Axi code. The body of an ordinary transaction is an Axi function that describes how to modify the ledger. Transactions also have some additional metadata which won't concern us here.

D.2: The resource lifecycle

A new resource type is created by a transaction that deploys a package of Axi code that contains the definition of this resource type. The only exceptions are built-in resource types, including Gas, the native coin used to pay transaction fees, which already exist at the Genesis block. Because packages live on-chain forever and cannot be deleted, a resource type, once it comes to life, will also exist forever.

The life cycle of a particular resource, i.e. an element of some resource type, consists of three stages:

- **Creation**. At some point the resource is created, in accordance with the interface of the package that contains the definition of the resource's type. After creation, the resource is placed on the ledger in a cell on its own or as a part of a bigger resource, owned by an account designated by its address, or in a shared cell.

- **Transaction(s)**. The transactor, as the owner, can transfer her ownership of the resource freely. Resources in shared cells can be altered through the interface set by the defining package of the resource type of a shared cell.

- **Destruction (optional)**. If the defining package permits it, the resource might at some point be destroyed. Examples of destructible resources are stablecoins that are backed by an off-chain reserve, where a coin gets destroyed when its off-chain collateral is removed from the treasury, and a wrapper for an asset bridged from another blockchain, which gets destroyed when the external asset ceases to be bridged.

The most important part of the lifecycle, in which most resources will spend almost all their life, is the phase when they can participate in transactions. An (ordinary) transaction consists of an arbitrary Axi program that transforms the state of the ledger. This program, when executed, interacts with the ledger in three steps:

- **Withdraw**. First, resources are "withdrawn" from the ledger into the program. The ledger's ownership model ensures the safety of this step, for example by making sure that the program cannot withdraw resources that don't belong to the transactor.
- **Transform**. Second, the program transforms its resources by performing various computations, exchanging the resources for other resources and moving them around, etc. The safety of this step is guaranteed by the type system. The resources cannot get unlawfully destroyed or copied because the type system tracks usage, nonsensical operations cannot be performed on them, because the program is well-typed, they cannot get lost in the jungle of pattern matching, because exhaustiveness is checked, and so on.
- **Deposit**. Third, the program "deposits" the transformed resources back onto the ledger. The safety of this step is partially ensured by the type system, which checks that all resources held by the program are deposited back. However, since resources can be freely transferred, it's up to the transactor to make sure that they are transferred to the correct address and not to somebody else.

As we can see, the resource lifecycle is determined by two orthogonal but complementary forces: package and service interfaces and the ownership system. The creator of the module that defines a new resource type sets the interface for all uses of all resources of this type. For a service (i.e. a shared mutable cell), its creator through its interface defines how the service handles all its resources. For an owned cell, its owner determines how the ownership of the resource changes, with which services it will interact (and through which methods of these services), and what primitive operations (coming from the defining package) will be performed on the resource.

D.3: Ledger primitives

In this section we will see how Khalani's ledger looks like from Axi's perspective. To do this, we will discuss the primitive (i.e. built-in) types and functions that Axi programs use to interact with the ledger.

Example 7. *Axi primitives for interacting with the ledger*

```
primitive
  // Provides access to the ledger and transaction metadata.
  Ledger : Type1

  // Type of accounts, stored in account cells.
  Account : Type1

  // A unique address of a cell.
  Address : Type

  // A reference to an account cell or a shared cell.
  Ref : Type1 -> Type

  // Extract an address from a reference.
  addressof : forall (A : Type1), Ref A -> Address

  // Try to convert an address to a reference.
  referenceof : forall @(A : Type1),
    Address -> Ledger -> Option (Ref A) * Ledger

  // Return the address that signed the currently executed transaction.
  transactor : Ledger -> Ref Account * Ledger

  // Create a new cell owned by the specified account.
  new : forall (A : Type1), Ref Account -> Ledger -> Address * (A -> Ledger)

  // Destroy an existing cell owned by the transactor
```

```
// and move the cell's resources into the program.
delete : forall @(A : Type1), Address -> Ledger -> Option A * Ledger

// Create a new mutable shared cell.
share : forall (A : Type1), Ledger -> Ref A * (A -> Ledger)

// Load the contents of a shared cell and then store a new value.
update : forall (A : Type1), Ref A -> Ledger -> A * (A -> Ledger)

// Create a new immutable shared cell.
freeze : forall (A : Type), Ledger -> Address * (A -> Ledger)

// Read the contents of an immutable shared cell.
peek : forall @(A : Type), Address -> Ledger -> Option A * Ledger
```

The above code listing shows the most important Axi primitives for dealing with the ledger. `Ledger` is the type that represents (the current state of) the ledger. When we said that the body of an ordinary transaction is an "arbitrary Axi program that transforms the ledger", we meant precisely that it is a function of type `Ledger -> Ledger`.

Every function that interacts with the ledger takes (the current version of) the ledger as input and also returns (the new version of) the ledger as output. In general, the type of such a function is `A -> Ledger -> B * Ledger`, i.e. it takes a value of some type `A` and the ledger, and returns a value of some type `B` and the (updated) ledger.

The transaction body will, in general, be a composition of many such functions. It will get the initial input of type `Ledger` from the transaction context and then pass it to the first of these functions, which will in turn pass it to the next one, and so on. Because `Ledger` is a linear type, the type system guarantees that values of this type must be used exactly once and cannot be copied nor deleted. This means that the ledger is acted upon and then passed further and further until the transaction body returns the final modified version of it. This is how all interaction with the ledger is represented in code.

An example function that needs the ledger is `transactor`, which consumes the old ledger and returns a reference to the account which originated the current transaction (i.e. the transactor) together with a new ledger.

`Address` is the data type of addresses, including both accounts and cell addresses. `Account` is the type of addresses that are known to be addresses of accounts.

`Ref` is a type constructor that turns resource types into data types that represent "references" to cells that contain this type of resource. The problem we want to solve is that, when interacting with shared cells, we don't know whether a shared cell with the given address exists, so we would need to check that first and handle the possible errors, but doing that over and over is tedious. As a solution, we perform a kind of "handshake" with the cell using the function `referenceof` which, if a cell with the given address exists, returns a reference to it. In practice, `Ref A` is just a wrapper around an `Address` that points to a shared cell that holds a value of type `A`. Besides shared cells, we also use `Ref` with `Accounts`, which due to account abstraction also have their own cells. This is useful to make the type signature of `new` simpler.

The function `addressof` returns the address when given the reference. The function `referenceof`, when given an address, together with the ledger, returns an optional reference, together with the ledger. If such a cell doesn't exist, the result is `none` (together with the new ledger). If it does, the result is `some ref` (together with the new ledger), where `ref` is the reference to the cell.

`new` is a function that takes an account reference (the type argument is implicit), together with the ledger and creates a new cell on the ledger. It returns the address of the created cell together with a function of type `A -> Ledger`. This function is a "setter" for the cell – when it is given a value of type `A`, it sets the cell contents to this value and then returns the new ledger. Note that the type `A -> Ledger` is linear, so the type system makes sure that it is called exactly once. Because of this, the cell can't be left empty, as well as it cannot be overwritten right after being created. This is a pattern that we will also see when creating and updating the shared cells.

`delete` is a function that takes a type `A` (note that the symbol `@` marks that this type argument must be given explicitly) and a cell address, together with the ledger, and returns an optional `A`, together with the new ledger. If the transactor is not the owner of the cell, or the value stored in the cell is not of type `A`, then the operation fails and the result is `none` (together with the new ledger).

Otherwise the operation succeeds, the cell is destroyed and the result is `some a` (together with the new ledger), where `a` is the value that was stored in this cell.

`share` is a function that takes just the ledger (the type argument is implicit), creates a new mutable shared cell, and returns the reference to the created cell, together with a setter function that, when given a value of type `A`, sets the cell's contents to this value and returns the new ledger.

`update` is a function that takes a reference to a mutable shared cell, together with the ledger, and returns a value of some type `A` together with a setter function that, when given a value of type `A`, sets the cell's contents to this value and returns the new ledger. Because the type `A` is linear, we must use the returned value exactly once, and because the type `A -> Ledger` is also linear, we must call the setter with some `A`. In practice, this means that if the type `A` is abstract, everything we can do is modify the `A` we were given and then put it back into the cell by calling the setter.

`freeze` is a function that takes just the ledger (the data type argument `A` is implicit), creates a new immutable shared cell, and returns the address of the created cell, together with a setter function that, when given a value of type `A`, sets the cell's contents to this value and returns the new ledger.

`peek` is a function that takes a type `A` and an address, together with the ledger, reads the contants of an immutable shared cell under this address, and returns an optional `A`, together with the new ledger. If the address does not point to an immutable shared cell, or the value stored in that cell is not of type `A`, then the operation fails and the result is `none` (together with the new ledger). Otherwise the operation succeeds and the result is `some a` (together with the new ledger), where `a` is the data stored in the cell.

D.4: A longer example – donut shop

In this section, we will see a longer example that demonstrates a simple on-chain service – the donut shop. This example was inspired by a similar example from the Sui documentation.

Example 8. *Donut Shop*

```
module DonutShop where

// Exporting the types would allow public access
// to constructors and record updates which we don't want.
// Since the types are mentioned in the signatures
// they will be public but opaque.
export
  init
  buyDonut
  collectProfits
  getMyShopRef

// Donut - a simple purchasable object. It can be used at most once.
record Donut : Type? where
  serialNumber : Nat

// Donut shop - it knows its address, donut price,
// the amount of money it earned so far, and the
// serial number of the next donut to be sold.
record DonutShop : Type1 where
  thisShop : Address
  price : Nat
  balance : KhalaniCoin
  nextSerialNumber : Nat

// A token which allows the owner to collect profits from his shop.
record DonutShopOwnershipToken : Type1 where
  myShopRef : Ref DonutShop

// Create a new donut shop, with donut price set to `myPrice`.
init (myPrice : Nat) (ledger : Ledger)
: DonutShopOwnershipToken * Ledger
= let
    (shopRef, publishShopToLedger) = share ledger
    ownership : DonutShopOwnershipToken =
      record where myShopRef = shopRef
    shop : DonutShop = record where
```

```
38        thisShop = addressof shopRef
39        price = myPrice
40        balance = coinZero
41        nextSerialNumber = 0
42    in
43    (ownership, publishShopToLedger shop)
44
45  // Buy a donut from the shop. The outputs are: the donut
46  // (if successfully bought), the change and the updated shop.
47  buyDonut (payment : KhalaniCoin) (shop : DonutShop)
48  : Option Donut * KhalaniCoin * DonutShop
49  = let (splitResult, change) = coinSplit shop.price payment in
50    match splitResult with
51    | none => (none, change, shop)
52    | some paid =>
53      let
54        donutSerialNumber = shop.nextSerialNumber
55        donut : Donut = record where serialNumber = donutSerialNumber
56        newBalance = coinMerge shop.balance paid
57        newShop : DonutShop =
58          record shop where
59            balance = newBalance
60            nextSerialNumber = donutSerialNumber + 1
61      in
62      (some donut, change, newShop)
63
64  // Collect profits from the shop (succeeds only for the owner).
65  collectProfits (ownership : DonutShopOwnershipToken) (shop : DonutShop)
66  : KhalaniCoin * DonutShopOwnershipToken * DonutShop
67  = if addressof ownership.myShop == shop.thisShop then
68      let
69        profits = shop.balance
70        newShop = record shop where balance = coinZero
71      in
72      (profits, ownership, newShop)
73    else
74      (coinZero, ownership, shop)
75
76  // Read shop reference from an ownership token
77  getMyShopRef (ownership : DonutShopOwnershipToken)
78  : Ref DonutShop * DonutShopOwnershipToken
79  = (token.myShopRef, token)
```

The above listing shows the code of the module `DonutShop`. This module provides functions for creating, managing and interacting with donut shops, which will live on-chain as *services*. We use the name service to refer to a mutable shared cell which stores a value whose type is abstract, i.e. whose definition is not exported by the module that defines it. Because of this, the only way to interact with a service is to get it from the ledger, call some of its methods (i.e. functions from the module which defines it) and then put it back onto the ledger.

When the code above gets deployed onto the chain, the resulting package will effectively become a public shared library that everybody can use in their own code. However, to make the examples simpler, we suppress details related to packages and instead pretend that it's the module itself that gets deployed onto the chain.

First, on lines 7-11, we indicate which functions we want the module to export. These functions become the interface of the module through which other code interacts with types from this module. We export all the functions we define, but we don't export the types we define nor their constructors or destructors, because we want to keep all of them abstract, so that elements of the `DonutShop` type can become on-chain services.

On lines 14-15 we define the `Donut` type. Because of the annotation `: Type?`, `Donut` is an affine type, i.e. its elements must be used at most once. Every donut has a serial number – this guarantees that every shop has an infinite supply of donuts, but otherwise this detail isn't very relevant to the example.

On lines 20-24 we define the `DonutShop` type. A shop stores its own on-chain address (so that it can check if the person trying to collect its profit is its owner), its balance (the coins it earned so far from selling donuts), the price of a single donut and the serial number of the next donut it will sell. `DonutShop` is a linear type, so every shop must be used exactly once. This means that functions

which interact with the donut shop will take the shop as input and will return an updated shop as output.

On lines 27-28 we define the `DonutShopOwnershipToken` type. Such a token contains a reference to the on-chain cell in which the shop lives and acts as a "key" which allows the shop owner (and only him) to withdraw the profits made by the shop. We expect such ownership tokens to become a common programming pattern in Khalani. We will explain how these tokens work later.

On lines 31-43 we define the function `init`. It takes a donut price, together with the ledger, sets up a new donut shop on the ledger, and returns its ownership token, together with the new ledger. It works as follows. First, on line 34, we create a new mutable shared cell that will hold the shop. In return we get a reference and a setter. Then on lines 35-36 we define a new ownership token using the reference. Then on lines 37-41 we define a new shop using the reference's address. Finally, on line 43, we return the ownership token, together with the new ledger obtained by calling the setter `publishShopToLedger` on the new shop.

On lines 47-62 we define the function `buyDonut`. It takes a coin used as payment and a donut shop, and returns an optional donut (depending on whether the purchase succeeds or not), the change left after paying, and the updated donut shop. It works as follows. First, on line 49, we split the payment coin to see if there's enough of it to pay the donut's price. On line 50 we check if the split succeeded. If not (line 51), we return `none`, the change (which is equal to the original payment coin) and the unmodified shop. If the split succeeded (line 52), we create a new donut (line 55), compute the shop's new balance (line 56) and update the shop (lines 57-60) to reflect the new balance and new serial number. Finally, on line 62, we return the donut, the change and the updated shop.

Note that, even though on line 49 we access the `shop`'s `price` field, this does not count as a usage of `shop` – this is because the accessed field, `price`, is data and not a resource. We allow to access data fields of records without consuming the record in order to avoid lengthy decomposition and recomposition of the record every time we need to peek at some data. On line 56, when we access the `shop`'s `balance` field, which is a resource, it does count as a usage of `shop`. However, when on line 58 we use `shop` as part of the *record update syntax* to build the updated shop, this again does not count as a usage, because all the fields that still "remain" in `shop` are data fields. `newShop` has the same fields as `shop`, except for the fields that were overwritten on lines 59-60. Because on line 59 we provide the `balance` field, which was "withdrawn" from `shop` by the field access on line 56, the definition of `newShop` through record update of `shop` is valid and follows correct resource usage.

On lines 65-74 we define the function `collectProfits`. It takes an ownership token and a shop and returns coins, the new shop and the new token. It works as follows. On line 67 we check if the address of the ownership token's reference is the same as the address of the shop. If yes, on lines 69-70 we withdraw the profits from the shop and update the shop's balance to zero, and then on line 72 we return the profits, the ownership token and the updated shop. If the token's address is different from the shop's address, the operation is unauthorized, and in such a case on line 74 we return a zero-value coin, the onwership token and the original shop.

Last but not least, on lines 77-79 we define the function `getMyShopRef` which, when given an ownership token, returns a reference to the shop and the ownership token. This function is needed because, even though inside the `DonutShop` module the ownership token's reference can be accessed using record field access syntax, outside the module this is not the case (because the definition of `DonutShopOwnershipToken` is not exported). Thus, without `getMyShopRef`, the shop owner couldn't get a reference to his shop from his ownership token. This wouldn't be a fatal flaw, but only a mild annoyance.

These technical descriptions of the functions' inner workings might sound scary, but in practice things are quite simple. Anybody can call `init` to set up a new donut shop. In return he gets an ownership token, which allows collecting the shop's profits and also contains the shop's reference (and thus address), so that he remembers where his shop is located. Anybody can try to buy a donut from any donut shop by calling the `buyDonut` function. If the argument contains enough coins, he will get his donut and change. If not, he doesn't get a donut but gets his coins back. In both cases, he can continue his interaction with the shop. Finally, anybody with any donut shop ownership token can try to collect profits from any donut shop. If the token happens to be the correct token for the particular shop, he will get his profits. If not, he gets a zero coins. In both cases, he gets his ownership token back and can continue his interaction with the shop.

Now, why does the donut shop ownership token work? Why can't anybody steal the shop's

profits? First, the `DonutShopOwnershipToken` is abstract, i.e. its definition is not exported by the module, so the only way to create such a token is by calling `init`. Second, this type is linear, so that the ownership token cannot be copied or deleted (although it might be sent to a dummy address, becoming effectively lost forever). This ensures that on-chain there always exists a unique ownership token for every donut shop. Because every donut shop stores its own on-chain address, and this address cannot be changed either by moving the shop on-chain or by modifying the shop's `thisShop` field, the check that the `collectProfits` function performs is enough to identify whether the caller is the shop's rightful owner or not. This argument could in principle be turned into a theorem, but we leave this for further work.

Another interesting property of this design is that, although ownership tokens cannot be forged, copied nor deleted, they can be exchanged, bought and sold. This means, for example, that a donut shop owner could sell his shop for a lump sum, letting the buyer collect the profits over time. Thus, we can see the donut shop ownership token as a very simple kind of stock that can be traded OTC and pays dividends on demand, but all at once (the owner can't withdraw only a part of the profits). These stock-like properties were not explicitly designed by us when we were coming up with this example. Instead, they arose naturally from the "ownership token" programming pattern, abstract types and linear typing.

Example 9. *A transaction that sets up a new donut shop*

```
1  from DonutShop import
2    init
3
4  main
5  : Ledger -> Ledger
6  | ledger =>
7    let
8      (myAccountRef, ledger) = transactor ledger
9      price = 1000
10     (ownershipToken, ledger) = init price ledger
11     (tokenAddress, initOwnershipTokenCell) = new myAccountRef ledger
12     ledger = initOwnershipTokenCell ownershipToken
13   in
14   ledger
```

The above example shows a transaction that sets up a new donut shop. On lines 1-2, we import the `init` function from the `DonutShop` module. On lines 4-14 we define the `main` function, which describes how the transaction will modify the ledger once it is executed. On line 8 we ask for a reference to our own account (so that we can create new cells later). On line 10 we call `init` to set up a new donut shop in which each donut costs 1000 `KhalaniCoin`s. On line 11 we create a new cell owned by us that will hold the shop's ownership token. On line 12 we call the setter to put our token into this cell. Finally, on line 14, we return the ledger.

Example 10. *A transaction that buys a donut*

```
from DonutShop import
  DonutShop
  buyDonut

coinAddr
: Address
= 0x0664eebd187e0668e41e5d2236eaafe2d54dd9dbc76685e4ceac69c908c26446

shopAddr
: Address
= 0xf09785359712726892a2ec3b8b7cf460f0f6dd82e0bf5a7a8d2f2276035888cf

main
: Ledger -> Ledger
| ledger =>
  let
    (myAccountRef, ledger) = transactor ledger
    (optShopRef, ledger) = referenceof DonutShop shopAddr ledger
  in
  match optShopRef with
  | none => ledger
```

```
22   | some shopRef =>
23     let (optCoin, ledger) = delete KhalaniCoin coinAddr ledger in
24     match optCoin with
25     | none => ledger
26     | some paymentCoin =>
27       let
28         (shop, returnShop) = update shopRef ledger
29         (optDonut, changeCoin, shop) = buyDonut paymentCoin shop
30         ledger = returnShop shop
31         (newCoinAddr, initCoinCell) = new myAccountRef ledger
32         ledger = initCoinCell changeCoin
33       in
34       match optDonut with
35       | none => ledger
36       | some donut =>
37         let
38           (newDonutAddr, initDonutCell) = new myAccountRef ledger
39           ledger = initDonutCell donut
40         in
41         ledger
```

The above example shows a transaction that buys a donut. On lines 1-3 we import the types `Donut`, `DonutShop` and the function `buyDonut` from the `DonutShop` module. On lines 5-7, we define a constant that holds the address of our cell with coins that we will use to pay for the donut. On lines 9-11 we define a constant that holds the address of the donut shop from which we will buy the donut. On lines 13-41 we define the `main` function. On line 17 we ask for a reference to our own account. On line 18 we ask for a reference to the donut shop. On line 20 we check if there indeed is a donut shop under the given address. If not (line 21), then we finish by returning the ledger. If yes, we now have a reference to the shop (line 22). On line 23 we attempt to grab our coins from their cell and on line 24 we check if it succeeded. If not (for example because we made a typo in the address), we finish by returning the ledger (line 25). If yes, we now have the coins (line 26). On line 28 we get hold of the shop. On line 29 we call `buyDonut` on the shop, trying to pay with our coins. On line 30 we put the shop back onto its cell. On line 31 we create a new cell to hold the change that was left after paying for the donut. On line 32 we put the change into this cell. On line 34 we check if we bought the donut successfully. If not (for example because we didn't have enough money), then we finish by returning the ledger (line 35). If yes, we now have the donut (line 36). On line 38 we create a new cell owned by us that will hold the donut. On line 39 we put the donut into this cell. On line 41 we return the final ledger.

Example 11. *A transaction that tries to collect profits from a donut shop*

```
1  from DonutShop import
2    DonutShopOwnershipToken
3    collectProfits
4    getMyShopRef
5
6  ownershipTokenAddr
7  : Address
8  = 0x22bf7f5182a7d22decc11f2a8fd94e2c6834cebd2ab2b239cab598987454bc1b
9
10 main
11 : Ledger -> Ledger
12 | ledger =>
13   let
14     (myAccountRef, ledger) = transactor ledger
15     (optOwnershipToken, ledger) =
16       delete DonutShopOwnershipToken ownershipTokenAddr ledger
17   in
18   match optOwnershipToken with
19   | none => ledger
20   | some ownershipToken =>
21     let
22       (shopRef, ownershipToken) = getMyShopRef ownershipToken
23       (shop, returnShop) = update shopRef ledger
24       (profits, ownershipToken, shop) = collectProfits ownershipToken shop
25       ledger = returnShop shop
26       (newCoinAddr, initCoinCell) = new myAccountRef ledger
27       ledger = initCoinCell profits
```

```
28        (newTokenAddr, initOwnershipTokenCell) = new myAccountRef ledger
29        ledger = initOwnershipTokenCell ownershipToken
30      in
31      ledger
```

The above example shows a transaction that tries to collect profits from a donut shop. On lines 1-4 we import the necessary types and functions from the module `DonutShop`. On lines 6-8 we define a constant that holds the address of our donut shop ownership token. On lines 10-31 we define the `main` function. On line 14 we ask for a reference to our own account. On lines 15-16 we try to pick up our donut shop ownership token from the ledger. On line 18 we check if we got the token. If not (for example, because this cell holds something else and not our token), we finish by returning the ledger (line 19). If yes, we now have the ownership token (line 20). On line 22 we extract the shop reference from the token. On line 23 we pick up the shop from the ledger. On line 24 we call `collectProfits` on the shop. On line 25 we put the shop back onto the ledger. On line 26 we create a new cell owned by us that will hold the profits. On line 27 we put the profits into this cell. On line 28 we create a new cell owned by us that will hold our token (recall that the old cell was destroyed when we picked up the token). On line 29 we put the ownership token into this cell. On line 39 we return the final ledger.